

Verilog-RTL-Design and-Verification

FAROOOQ ABDULLA

Contents

VERILOG CODE	4
DATA-FLOW LEVEL	8
BASIC GATES	8
HALF ADDER.....	9
FULL ADDER	11
HALF SUBTRACTOR	12
FULL SUBTRACTOR	13
2:1 MUX	14
GATE OR STUCTURAL LEVEL.....	16
BASIC GATES	16
HALF ADDER.....	17
FULL ADDER	18
HALF SUBTRACTOR	20
FULL SUBTRACTOR	21
2:1 MUX	22
SWITCH-LEVEL	23
BASIC GATES	23
BEHAVIORAL OR ALGORITHMIC LEVEL	25
BASIC GATES	25
HALF ADDER.....	27
FULL ADDER	28
HALF SUBTRACTOR	29
FULL SUBTRACTOR	31
2:1 MUX	32
FOUR BIT COMPARATOR.....	33
4-BIT RIPPLE CARRY ADDER.....	35
LATCHES	37
JK Latch	37
SR Latch.....	39
D Latch.....	41
T Latch	42

FLIPFLOPS	44
SR Flipflop.....	44
D Flipflop.....	46
JK Flipflop	47
T Flipflop	49
SHIFT REGISTERS	51
SISO.....	51
SIPO.....	53
PIPO.....	55
PISO.....	56
COUNTERS	58
4-BIT UP COUNTER	58
4-BIT DOWN COUNTER	60
4-BIT UP/DOWN COUNTER	61
4-BIT RING COUNTER	63
4-BIT JOHNSON COUNTER.....	64
DECADE COUNTER	66
4-BIT RIPPLE COUNTER.....	67
NUMERICALS.....	69
SUM OF NUMBERS	69
SQUARE OF NUMBER	70
FACTORIAL OF A NUMBER.....	71
FIBANOCCHI SERIES.....	73

VERILOG CODE

Verilog is both a behavioral and a structural language. Internals of each module can be defined at four levels of abstraction, depending on the needs of the design. The module behaves identically with the external environment irrespective of the level of abstraction at which the module is described. The internals of the module are hidden from the environment. Thus, the level of abstraction to describe a module can be changed without any change in the environment. These levels will be studied in detail in separate chapters later in the book. The levels are defined below.

Behavioral or algorithmic level

This is the highest level of abstraction provided by Verilog HDL. A module can be implemented in terms of the desired design algorithm without concern for the hardware implementation details. Designing at this level is very similar to C programming.

```
module half_adder_behavioral(input a, input b, output reg sum, output reg carry);
    always @(*) begin
        sum = a ^ b;
        carry = a & b;
    end
endmodule
```

Dataflow level

At this level the module is designed by specifying the data flow. The designer is aware of how data flows between hardware registers and how the data is processed in the design.

```
module half_adder_dataflow(input a, input b, output sum, output carry);
    assign sum = a ^ b;
    assign carry = a & b;
endmodule
```

Gate or Structural level

The module is implemented in terms of logic gates and interconnections between these gates. Design at this level is similar to describing a design in terms of a gate-level logic diagram.

```
module half_adder_gate(input a, input b, output sum, output carry);
    xor(sum, a, b);
    and(carry, a, b);
endmodule
```

Switch level

This is the lowest level of abstraction provided by Verilog. A module can be implemented in terms of switches, storage nodes, and the interconnections between them. Design at this level requires knowledge of switch-level implementation details.

```

module half_adder_switch(input a, input b, output sum, output carry);
    nmos(carry, b, a);

    wire w1, w2;
    nmos(w1, a, b);
    nmos(w2, b, a);
    assign sum = w1 | w2;
endmodule

```

Verilog allows the designer to mix and match all four levels of abstractions in a design. In the digital design community, the term register transfer level (RTL) is frequently used for a Verilog description that uses a combination of behavioral and dataflow constructs and is acceptable to logic synthesis tools. If a design contains four modules, Verilog allows each of the modules to be written at a different level of abstraction. As the design matures, most modules are replaced with gate-level implementations. Normally, the higher the level of abstraction, the more flexible and technology independent the design. As one goes lower toward switch-level design, the design becomes technology dependent and inflexible. A small modification can cause a significant number of changes in the design. Consider the analogy with C programming and assembly language programming. It is easier to program in a higher-level language such as C. The program can be easily ported to any machine. However, if you design at the assembly level, the program is specific for that machine and cannot be easily ported to another machine.

Here is a comprehensive guide to operators, keywords, and identifiers used in Verilog HDL, which are foundational for digital design and coding in Verilog.

Operators in Verilog

Verilog supports several categories of operators:

- **Arithmetic Operators:**
+ (addition), - (subtraction), * (multiplication), / (division), % (modulus), ** (exponentiation).
- **Logical Operators:**
&& (logical AND), || (logical OR), ! (logical NOT).
- **Bitwise Operators:**
& (bitwise AND), | (bitwise OR), ^ (bitwise XOR), ~ (bitwise NOT), ~& (bitwise NAND), ~| (bitwise NOR), ~^/^~ (bitwise XNOR).
- **Reduction Operators:**
Same symbols as bitwise, but operate across all bits of the operand (&, |, ^, ~&, ~|, ~^).
- **Relational Operators:**
< (less than), > (greater than), <= (less than or equal), >= (greater than or equal).
- **Equality Operators:**
== (equal), != (not equal), === (case equality), !== (case inequality).

- **Shift Operators:**
<< (left shift), >> (right shift), <<< (arithmetic left shift), >>> (arithmetic right shift).
- **Conditional Operator:**
?: (ternary, conditional, if-else).
- **Concatenation & Replication:**
{ } (concatenate), { { } } (repeat/replicate).
- **Assignment:**
= (blocking assignment), <= (non-blocking assignment).
- **Miscellaneous:**
-> (event trigger), @ (event control).

Verilog Keywords

Some essential Verilog HDL keywords include:

- **Data Types & Declarations:**
module, input, output, inout, wire, reg, integer, real, parameter, localparam
- **Control & Flow:**
always, initial, if, else, case, casex, casez, for, while, repeat, forever
- **Procedures:**
function, task, begin, end
- **Timing & Events:**
#, @, posedge, negedge, wait, delay
- **Synthesis & Simulation:**
assign, deassign, force, release
- **Hierarchy & Structure:**
endmodule, primitive, interface, generate, endgenerate
- **Compiler Directives:**
'include, 'define, 'if def, 'endif, 'timescale

Identifiers in Verilog

- **Allowed Characters:**
Identifiers must start with an alphabetic character (A-Z, a-z) or an underscore (_) and can be followed by any combination of letters, digits (0-9), and underscores.
- **Case Sensitivity:**
Verilog is case-sensitive, so *signal* and *Signal* are distinct identifiers.
- **Length:**
Identifiers can be of any length, but tool-specific limits may apply.

- **Reserved Words:**
Verilog keywords (e.g., module, input, always) cannot be used as identifiers unless escaped.
- **Escaped Identifiers:**
Identifiers can begin with a backslash \ to include otherwise illegal characters or reserved words. The escaped identifier continues until the next whitespace.
Example: \1d_signal or \reset*
- **System Identifiers:**
Identifiers starting with a dollar sign \$ are reserved for system tasks and functions like \$display or \$finish.

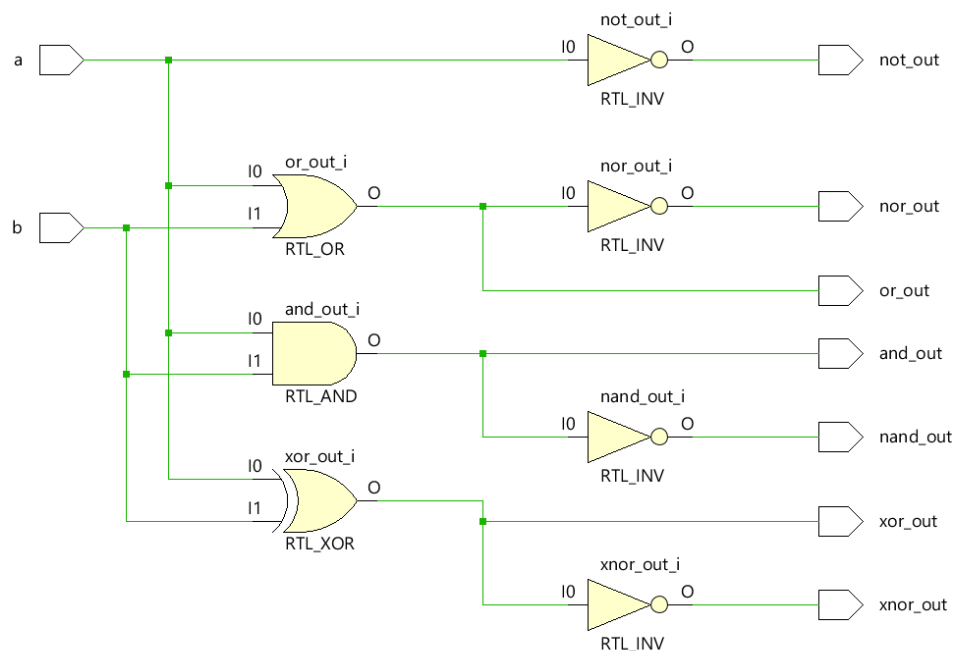
DATA-FLOW LEVEL

BASIC GATES

```
module Basic_gate(  
  input a, b,  
  output not_out, and_out, nand_out, or_out, nor_out, xor_out, xnor_out);  
  assign not_out = ~a;  
  assign and_out = a & b;  
  assign nand_out = ~(a & b);  
  assign or_out = a | b;  
  assign nor_out = ~(a | b);  
  assign xor_out = a ^ b;  
  assign xnor_out = ~(a ^ b);  
endmodule
```

```
module Basic_Gates_TB;  
  reg a, b;  
  wire not_out, and_out, nand_out, or_out, nor_out, xor_out, xnor_out;  
  Basic_gates uut  
  (.a(a), .b(b), .not_out(not_out), .and_out(and_out), .nand_out(nand_out), .or_out(or_out),  
  .nor_out(nor_out), .xor_out(xor_out), .xnor_out(xnor_out));  
  
  initial begin  
    a = 0; b = 0;  
    #10;  
    a = 0; b = 1;  
    #10;  
    a = 1; b = 0;  
    #10;  
    a = 1; b = 1; #10;  
    $display("%b %b| %b %b %b %b %b %b %b", a, b, not_out, and_out, nand_out,  
    or_out, nor_out, xor_out, xnor_out);  
  
    $finish;  
  end  
endmodule
```


Name	Value	0 ns	5 ns	10 ns	15 ns	20 ns	25 ns	30 ns	35 ns	40 ns
a	1									
b	1									
not_out	0									
and_out	1									
nand_out	0									
or_out	1									
nor_out	0									
xor_out	0									
xnor_out	1									



HALF ADDER

```

module Half_adder(input a,b,output s,c);
    assign s = a ^ b;
    assign c = a & b;
endmodule

```

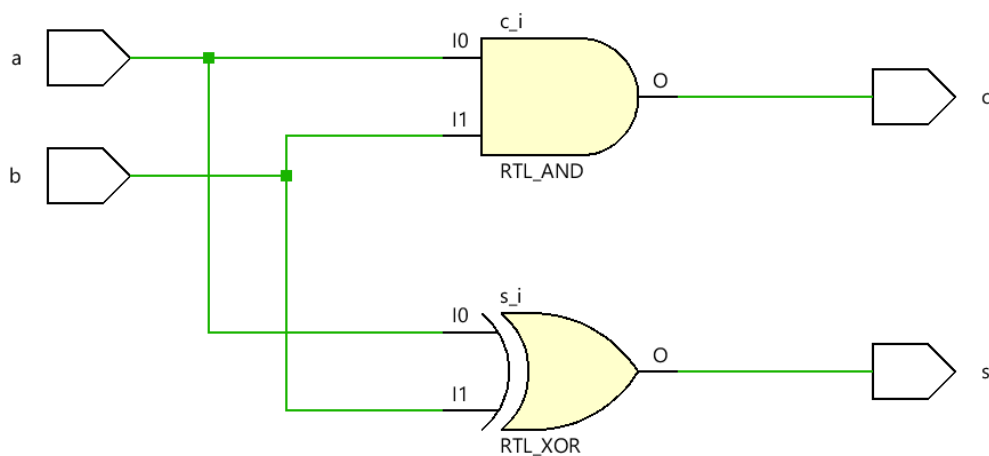
```

module Half_adder_TB;
    reg a, b;
    wire s, c;
    Half_adder uut (.a(a),.b(b),.s(s),.c(c));
    initial begin
        a = 0; b = 0;
        #10;
        a = 0; b = 1;
        #10;
        a = 1; b = 0;
        #10;
        a = 1; b = 1;
        #10;
        $display("%b %b|%b %b", a, b, s, c);

        $finish;
    end
endmodule

```

Name	Value	0 ns	10 ns	20 ns	30 ns	40 ns
a	1					
b	1					
s	0					
c	1					



FULL ADDER

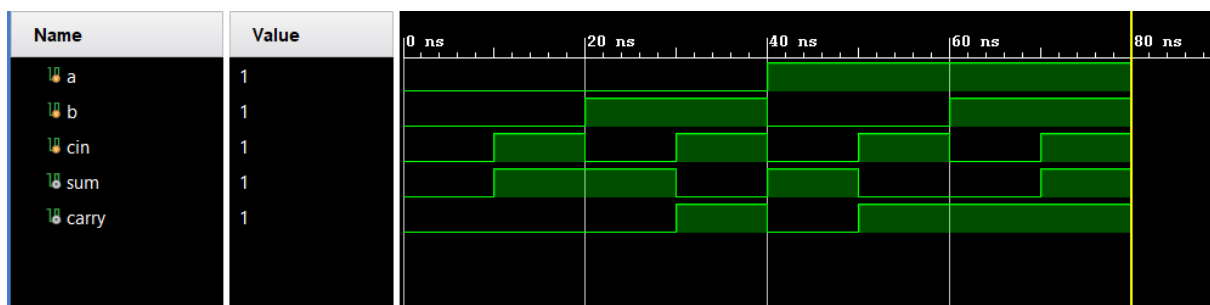
```
module Full_Adder(input a,b,cin,output sum, carry);
    assign sum  = a ^ b ^ cin;
    assign carry = (a & b) | (b & cin) | (cin & a);
endmodule
```

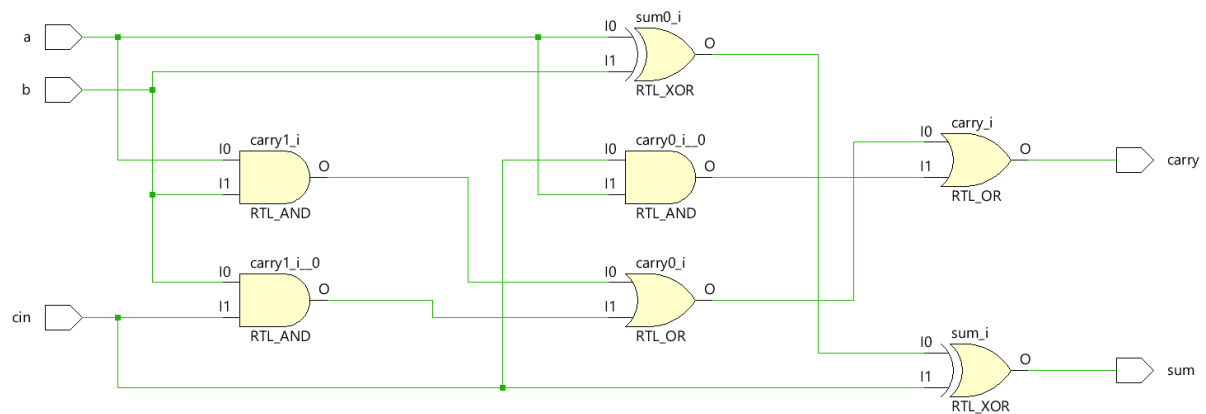
```
module Full_Adder_TB;

    reg a, b, cin;
    wire sum, carry;

    Full_Adder uut (.a(a),.b(b),.cin(cin),.sum(sum),.carry(carry));

    initial begin
        a = 0; b = 0; cin = 0; #10;
        a = 0; b = 0; cin = 1; #10;
        a = 0; b = 1; cin = 0; #10;
        a = 0; b = 1; cin = 1; #10;
        a = 1; b = 0; cin = 0; #10;
        a = 1; b = 0; cin = 1; #10;
        a = 1; b = 1; cin = 0; #10;
        a = 1; b = 1; cin = 1; #10;
        $display("%b %b %b | %b    %b", a, b, cin, sum, carry);
        $finish;
    end
endmodule
```

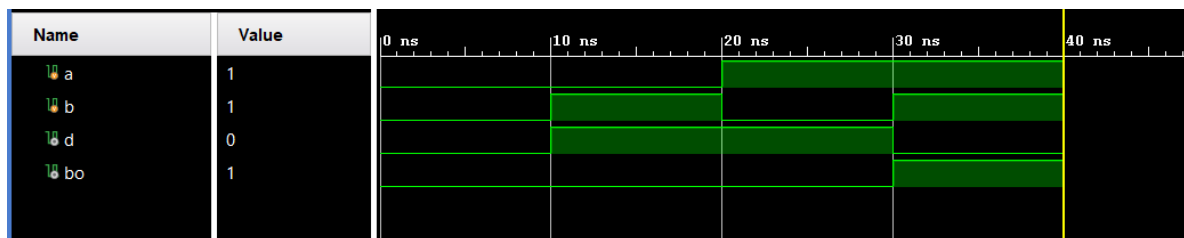
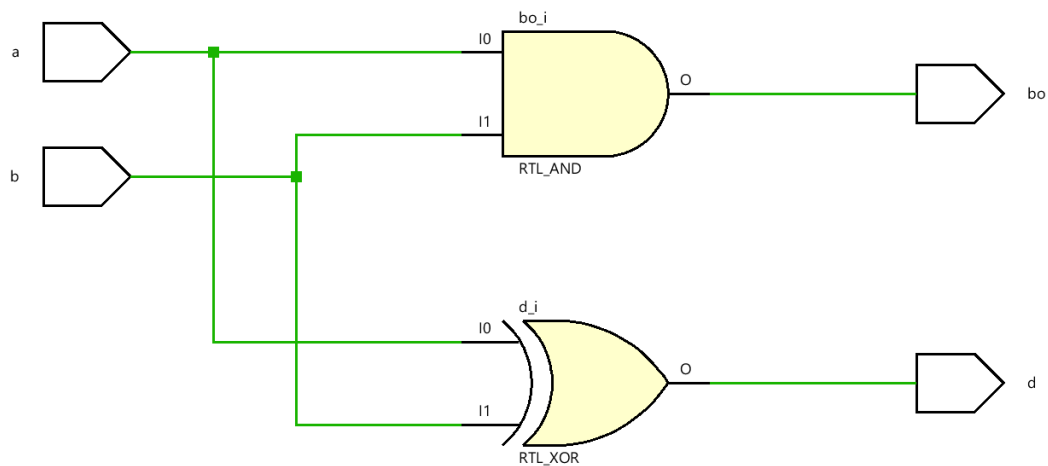




HALF SUBTRACTOR

```
module Half_Subtractor(input a,b,output d,bo);
    assign d = a ^ b;
    assign bo = a & b;
endmodule
```

```
module Half_Subtractor_TB;
    reg a, b;
    wire d, bo;
    Half_Subtractor uut (.a(a),.b(b),.d(d),.bo(bo));
    initial begin
        a = 0; b = 0; #10;
        a = 0; b = 1; #10;
        a = 1; b = 0; #10;
        a = 1; b = 1; #10;
        $display("%b %b|%b %b", a, b, d, bo);
        $finish;
    end
endmodule
```



FULL SUBTRACTOR

```
module Full_subtractor(input a,b,cin,output d, bo);
    assign d = a ^ b ^ cin;
    assign bo = (~a & b) | (b & cin) | (cin & ~a);
endmodule
```

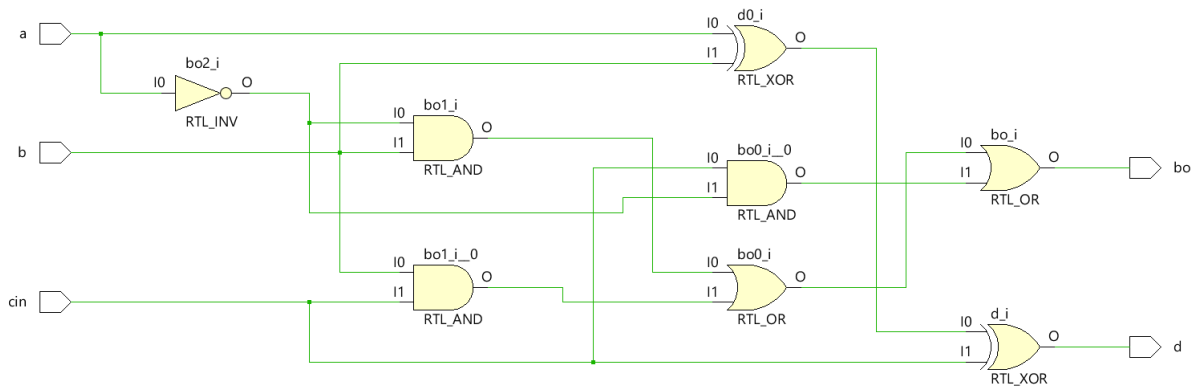
```
module Full_Subtractor_TB;
    reg a, b, cin;
    wire d,bo;

    Full_subtractor uut (.a(a),.b(b),.cin(cin),.d(d),.bo(bo));
```

```

initial begin
    a = 0; b = 0; cin = 0; #10;
    a = 0; b = 0; cin = 1; #10;
    a = 0; b = 1; cin = 0; #10;
    a = 0; b = 1; cin = 1; #10;
    a = 1; b = 0; cin = 0; #10;
    a = 1; b = 0; cin = 1; #10;
    a = 1; b = 1; cin = 0; #10;
    a = 1; b = 1; cin = 1; #10;
    $display("%b %b %b | %b      %b", a, b, cin, d, bo);
    $finish;
end
endmodule

```



Name	Value	0 ns	10 ns	20 ns	30 ns	40 ns	50 ns	60 ns	70 ns	80 ns
a	1									
b	1									
cin	1									
d	1									
bo	1									

2:1 MUX

```

module mux2to1(input a,b,sel,output y);
    assign y = sel ? b : a;
endmodule

```

```

module mux2to1_TB;

    reg a, b, sel;
    wire y;

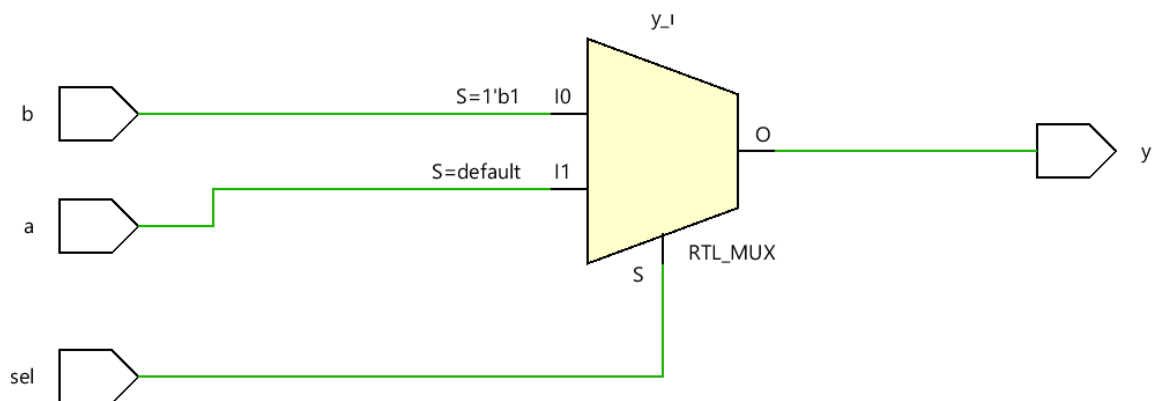
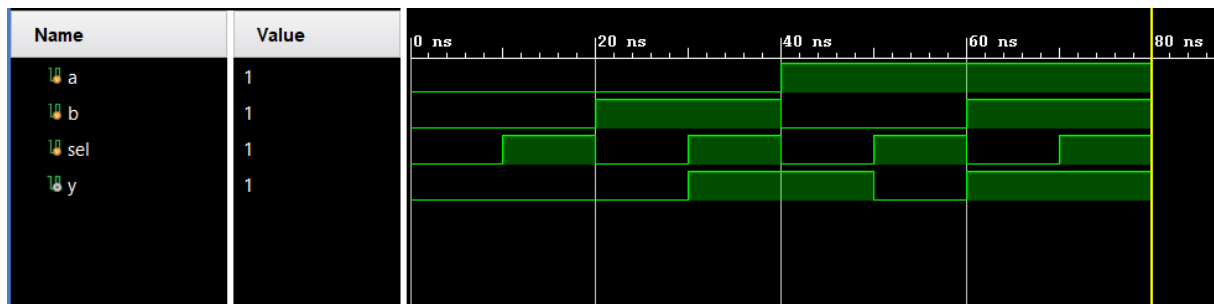
    mux2to1 uut (.a(a),.b(b),.sel(sel),.y(y));

    initial begin

        a = 0; b = 0; sel = 0; #10;
        a = 0; b = 0; sel = 1; #10;
        a = 0; b = 1; sel = 0; #10;
        a = 0; b = 1; sel = 1; #10;
        a = 1; b = 0; sel = 0; #10;
        a = 1; b = 0; sel = 1; #10;
        a = 1; b = 1; sel = 0; #10;
        a = 1; b = 1; sel = 1; #10;
        $display("%b %b %b | %b", a, b, sel, y);

        $finish;
    end
endmodule

```



GATE OR STRUCTURAL LEVEL

BASIC GATES

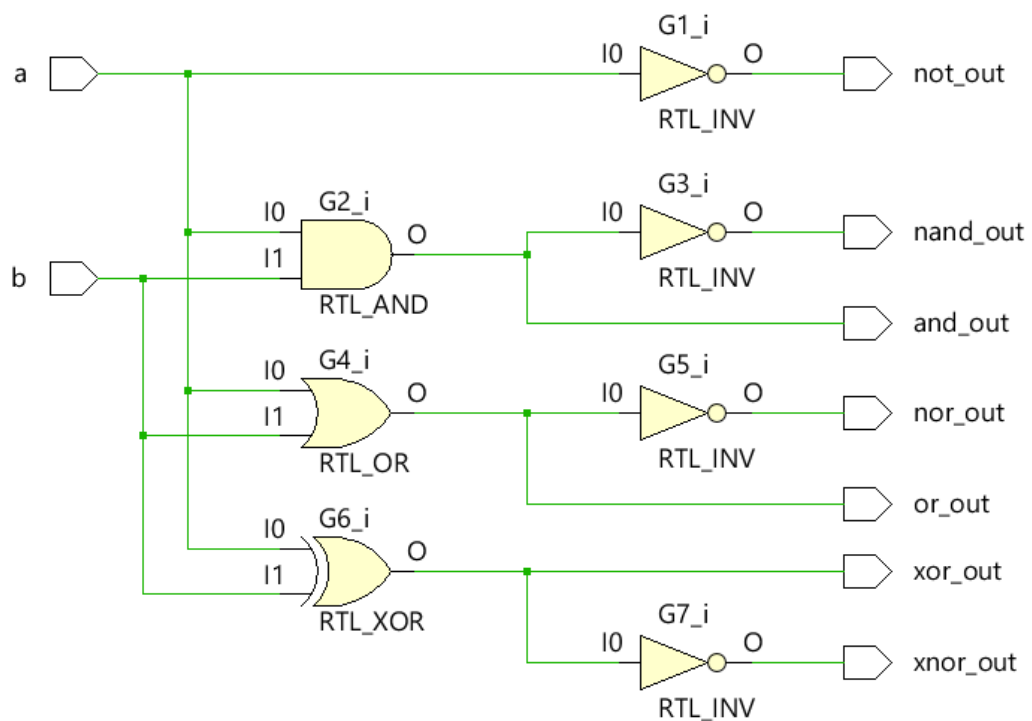
```
module Basic_Gates(input a,b,output
not_out,and_out,nand_out,or_out,nor_out,xor_out,xnor_out);
    not G1 (not_out, a);
    and G2 (and_out, a, b);
    nand G3 (nand_out, a, b);
    or G4 (or_out, a, b);
    nor G5 (nor_out, a, b);
    xor G6 (xor_out, a, b);
    xnor G7 (xnor_out, a, b);
endmodule
```

```
module Basic_Gates_TB;
    reg a, b;
    wire not_out, and_out, nand_out, or_out, nor_out, xor_out, xnor_out;

    Basic_Gates uut
    (.a(a),.b(b),.not_out(not_out),.and_out(and_out),.nand_out(nand_out),.or_out(or
_out),
    .nor_out(nor_out),.xor_out(xor_out),.xnor_out(xnor_out));

    initial begin
        a = 0; b = 0;
        #10;
        a = 0; b = 1;
        #10;
        a = 1; b = 0;
        #10;
        a = 1; b = 1; #10;
        $display("%b %b|%b %b %b %b %b %b", a, b, not_out, and_out, nand_out,
or_out, nor_out, xor_out, xnor_out);
        $finish;
    end
endmodule
```


Name	Value	0 ns	10 ns	20 ns	30 ns	40 ns
a	1					
b	1					
not_out	0					
and_out	1					
nand_out	0					
or_out	1					
nor_out	0					
xor_out	0					
xnor_out	1					



HALF ADDER

```

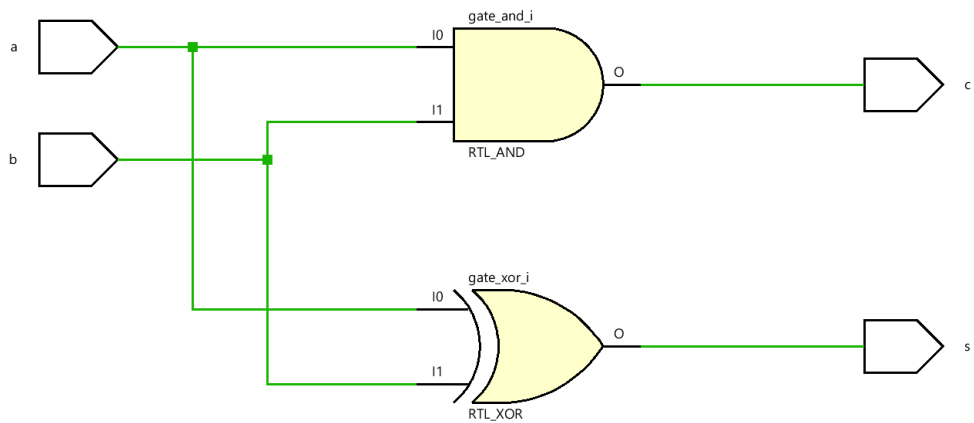
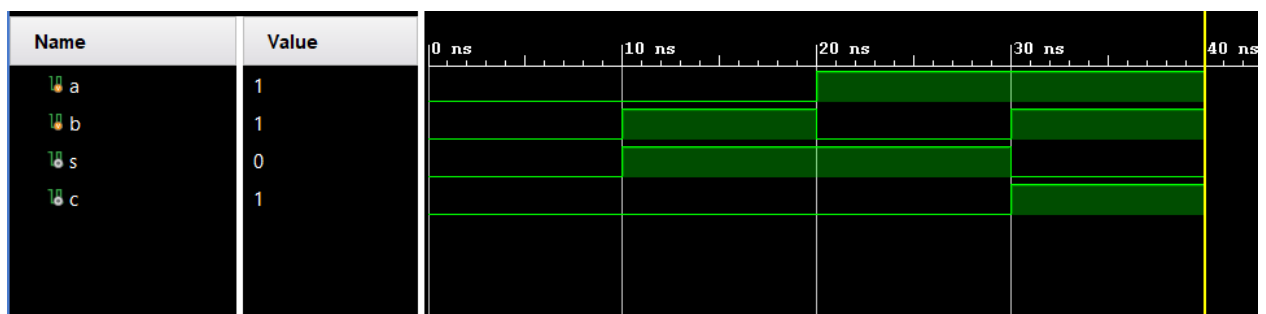
module Half_Adder(input a,b,output s,c);
    xor gate_xor(s, a, b);
    and gate_and(c, a, b);
endmodule

```

```

module Half_adder_TB;
  reg a, b;
  wire s, c;
  Half_Adder uut (.a(a),.b(b),.s(s),.c(c));
  initial begin
    a = 0; b = 0; #10;
    a = 0; b = 1; #10;
    a = 1; b = 0; #10;
    a = 1; b = 1; #10;
    $display("%b %b|%b %b", a, b, s, c);
    $finish;
  end
endmodule

```



FULL ADDER

```

module Full_Adder(input a,b,cin,output sum,carry);

  wire w1, w2, w3, w4;
  xor (w1, a, b);
  xor (sum, w1, cin);

```

```

    and (w2, a, b);
    and (w3, b, cin);
    and (w4, cin, a);
    or (carry, w2, w3, w4);
endmodule

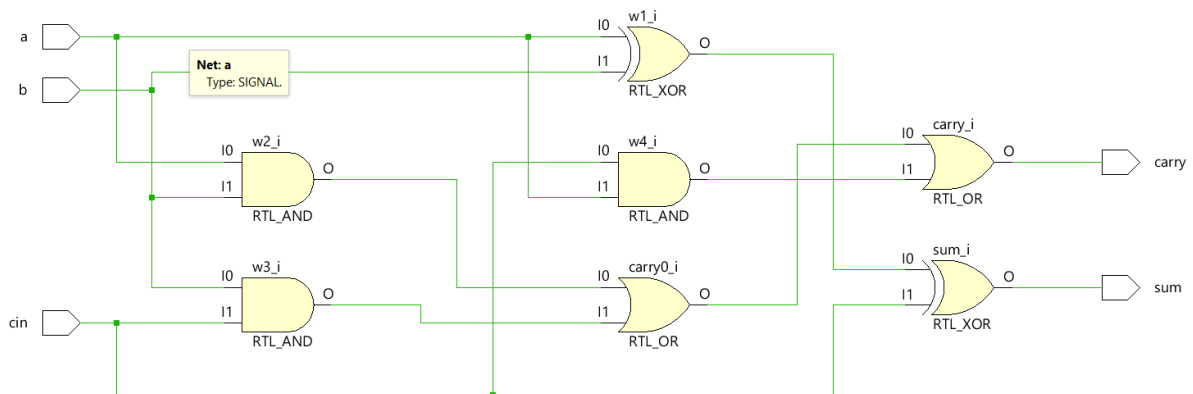
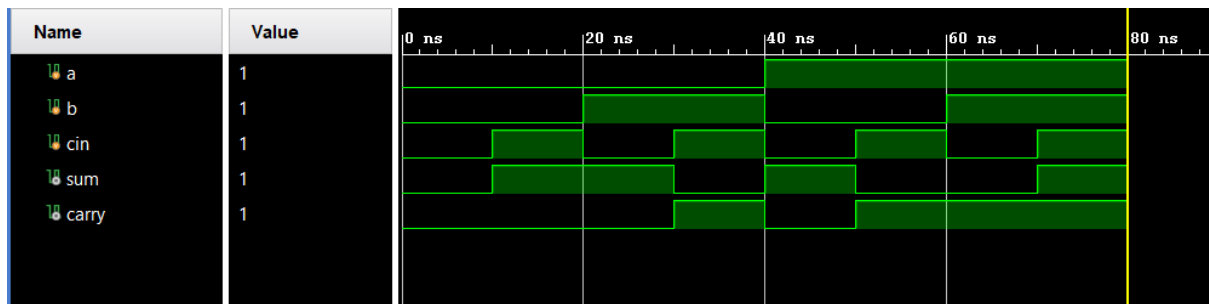
```

```

module Full_Adder_TB;

    reg a, b, cin;
    wire sum, carry;
    Full_Adder uut (.a(a),.b(b),.cin(cin),.sum(sum),.carry(carry));
    initial begin
        a = 0; b = 0; cin = 0; #10;
        a = 0; b = 0; cin = 1; #10;
        a = 0; b = 1; cin = 0; #10;
        a = 0; b = 1; cin = 1; #10;
        a = 1; b = 0; cin = 0; #10;
        a = 1; b = 0; cin = 1; #10;
        a = 1; b = 1; cin = 0; #10;
        a = 1; b = 1; cin = 1; #10;
        $display("%b %b %b | %b      %b", a, b, cin, sum, carry);
        $finish;
    end
endmodule

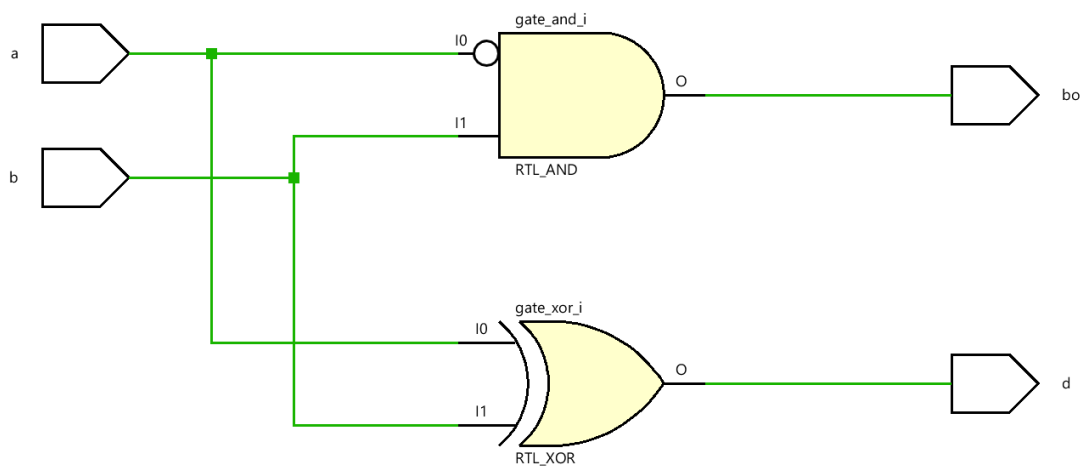
```



HALF SUBTRACTOR

```
module Half_Subtractor(input a,b,output d,bo);  
    xor gate_xor(d, a, b);  
    and gate_and(bo, ~a, b);  
endmodule
```

```
module Half_adder_TB;  
    reg a, b;  
    wire d, bo;  
    Half_Subtractor uut (.a(a),.b(b),.d(d),.bo(bo));  
    initial begin  
        a = 0; b = 0; #10;  
        a = 0; b = 1; #10;  
        a = 1; b = 0; #10;  
        a = 1; b = 1; #10;  
        $display("%b %b|%b %b", a, b, d, bo);  
        $finish;  
    end  
endmodule
```

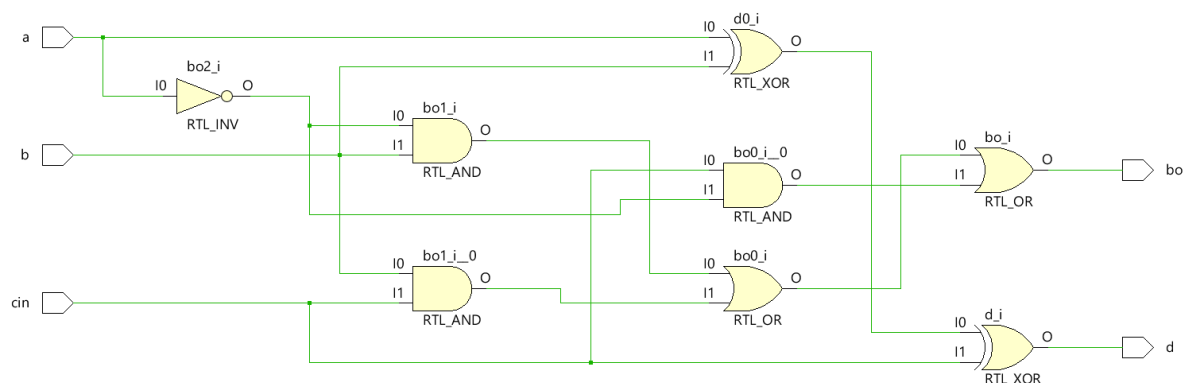


Name	Value	0 ns	5 ns	10 ns	15 ns	20 ns	25 ns	30 ns	35 ns	40 ns
a	1	0	0	1	1	0	0	1	1	0
b	1	0	1	0	0	1	1	0	0	1
d	0	0	1	1	0	1	0	0	1	1
bo	0	0	0	0	1	0	0	0	0	1

FULL SUBTRACTOR

```
module Full_Subtractor(input a,b,cin,output d,bo);  
  
    wire w1, w2, w3, w4;  
    xor (w1, a, b);  
    xor (d, w1, cin);  
    and (w2, ~a, b);  
    and (w3, b, cin);  
    and (w4, cin, ~a);  
    or (bo, w2, w3, w4);  
endmodule
```

```
module Full_Subtractor_TB;  
    reg a, b, cin;  
    wire d,bo;  
    Full_Subtractor uut (.a(a),.b(b),.cin(cin),.d(d),.bo(bo));  
    initial begin  
        a = 0; b = 0; cin = 0; #10;  
        a = 0; b = 0; cin = 1; #10;  
        a = 0; b = 1; cin = 0; #10;  
        a = 0; b = 1; cin = 1; #10;  
        a = 1; b = 0; cin = 0; #10;  
        a = 1; b = 0; cin = 1; #10;  
        a = 1; b = 1; cin = 0; #10;  
        a = 1; b = 1; cin = 1; #10;  
        $display("%b %b %b | %b    %b", a, b, cin, d, bo);  
        $finish;  
    end  
endmodule
```



Name	Value	0 ns	10 ns	20 ns	30 ns	40 ns	50 ns	60 ns	70 ns	80 ns	90 ns
a	1										
b	1										
cin	1										
d	1										
bo	1										

2:1 MUX

```

module mux2to1(input a,b,sel,output y);

    wire sel_n, w1, w2;

    not(sel_n, sel);
    and(w1, a, sel_n);
    and(w2, b, sel);
    or(y, w1, w2);
endmodule

```

```

module mux2to1_TB;

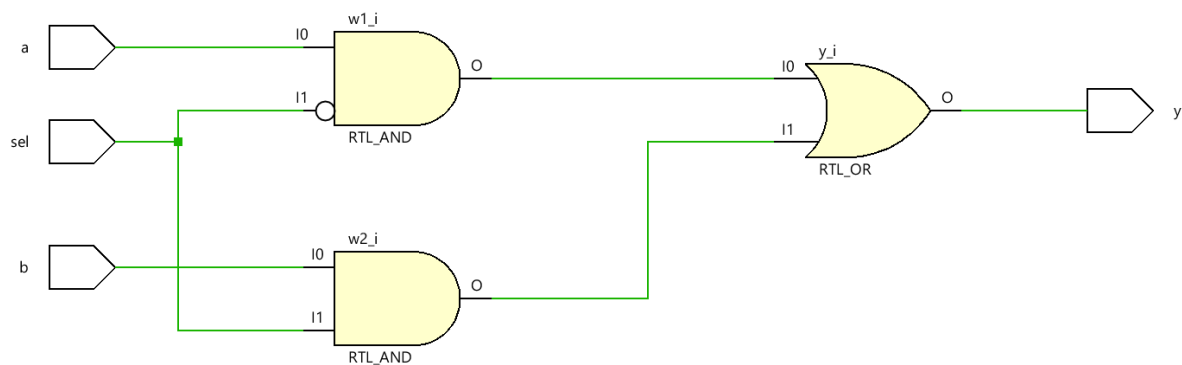
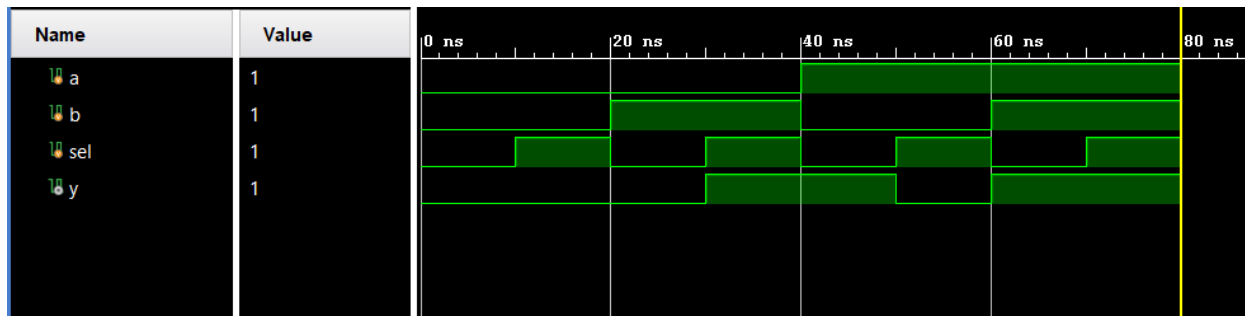
    reg a, b, sel;
    wire y;

    mux2to1 uut (.a(a),.b(b),.sel(sel),.y(y));

    initial begin
        a = 0; b = 0; sel = 0; #10;
        a = 0; b = 0; sel = 1; #10;
        a = 0; b = 1; sel = 0; #10;
        a = 0; b = 1; sel = 1; #10;
        a = 1; b = 0; sel = 0; #10;
        a = 1; b = 0; sel = 1; #10;
        a = 1; b = 1; sel = 0; #10;
        a = 1; b = 1; sel = 1; #10;
        $display("%b %b %b | %b", a, b, sel, y);

        $finish;
    end
endmodule

```



SWITCH-LEVEL

BASIC GATES

```

module Basic_gates (input a, b,output
not_out,and_out,nand_out,or_out,nor_out,xor_out,xnor_out);
    supply1 VDD;
    supply0 GND;

    // NOT gate
    pmos p_not(not_out, VDD, a);
    nmos n_not(not_out, GND, a);

    // NAND gate
    wire nand_pullup;
    wire nand_pmos_net;

    pmos p_nand1(nand_pullup, VDD, a);
    pmos p_nand2(nand_pullup, VDD, b);
  
```

```

wire nand_nmos_net;
nmos n_nand1(nand_nmos_net, GND, b);
nmos n_nand2(nand_pullup, nand_nmos_net, a);

assign nand_out = nand_pullup;

// AND gate = NOT(NAND)
wire and_net;
pmos p_and(and_net, VDD, nand_pullup);
nmos n_and(and_net, GND, nand_pullup);
assign and_out = and_net;

// NOR gate
wire nor_pullup;
pmos p_nor1(nor_pullup, VDD, a);
pmos p_nor2(nor_pullup, VDD, b);

wire nor_nmos_net;
nmos n_nor1(nor_nmos_net, GND, b);
nmos n_nor2(nor_pullup, nor_nmos_net, a);
assign nor_out = nor_pullup;

// OR gate = NOT(NOR)
wire or_net;
pmos p_or(or_net, VDD, nor_pullup);
nmos n_or(or_net, GND, nor_pullup);
assign or_out = or_net;

// XOR gate (simplified using gates)
wire na, nb, a_and_nb, b_and_na, xor_temp;
not (na, a);
not (nb, b);
and (a_and_nb, a, nb);
and (b_and_na, b, na);
or (xor_temp, a_and_nb, b_and_na);
assign xor_out = xor_temp;

// XNOR = NOT(XOR)
wire xnor_temp;
pmos p_xnor(xnor_temp, VDD, xor_temp);
nmos n_xnor(xnor_temp, GND, xor_temp);
assign xnor_out = xnor_temp;

endmodule

```

```

module Basic_Gates_TB;

```

```

    reg a, b;

```



```

wire not_out, and_out, nand_out, or_out, nor_out, xor_out, xnor_out;

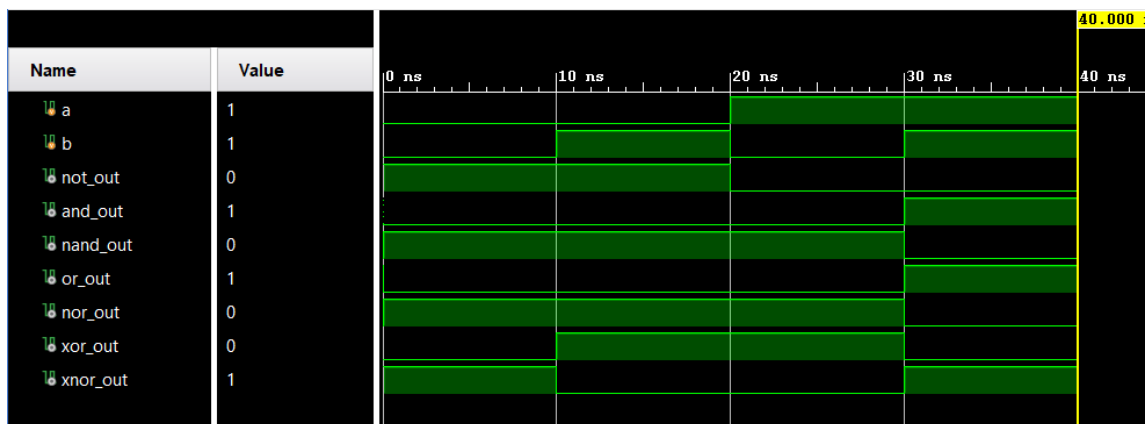
Basic_gates uut
(.a(a),.b(b),.not_out(not_out),.and_out(and_out),.nand_out(nand_out),.or_out(or_out),
.nor_out(nor_out),.xor_out(xor_out),.xnor_out(xnor_out));

initial begin
    a = 0; b = 0;
    #10;
    a = 0; b = 1;
    #10;
    a = 1; b = 0;
    #10;
    a = 1; b = 1; #10;
    $display("%b %b| %b %b %b %b %b %b", a, b, not_out, and_out, nand_out,
or_out, nor_out, xor_out, xnor_out);

    $finish;
end

endmodule

```



BEHAVIORAL OR ALGORITHMIC LEVEL

BASIC GATES

```

module Basic_Gates(input a, b,output reg
not_out,and_out,nand_out,or_out,nor_out,xor_out,xnor_out);
    always @(*) begin
        not_out  = ~a;
        and_out  = a & b;
        nand_out = ~(a & b);
        or_out   = a | b;
    end
endmodule

```

```

        nor_out  = ~(a | b);
        xor_out  = a ^ b;
        xnor_out = ~(a ^ b);
    end
endmodule

```

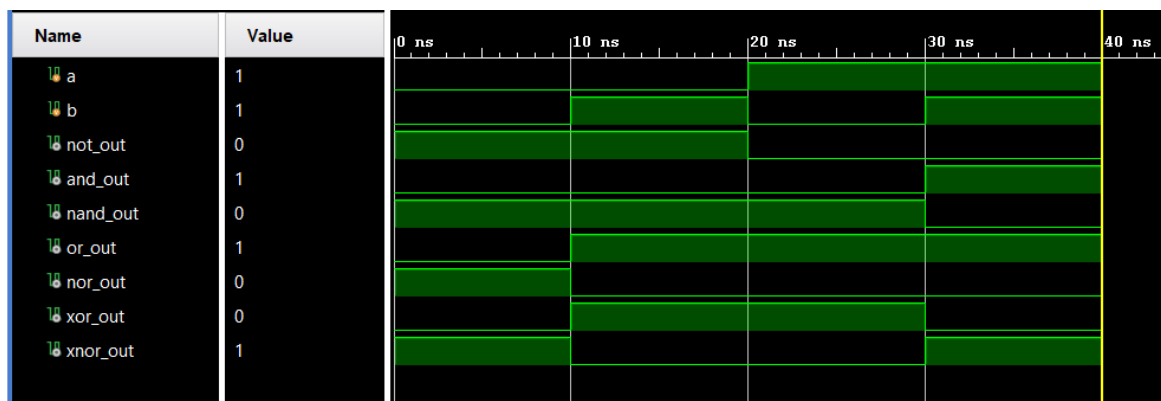
```

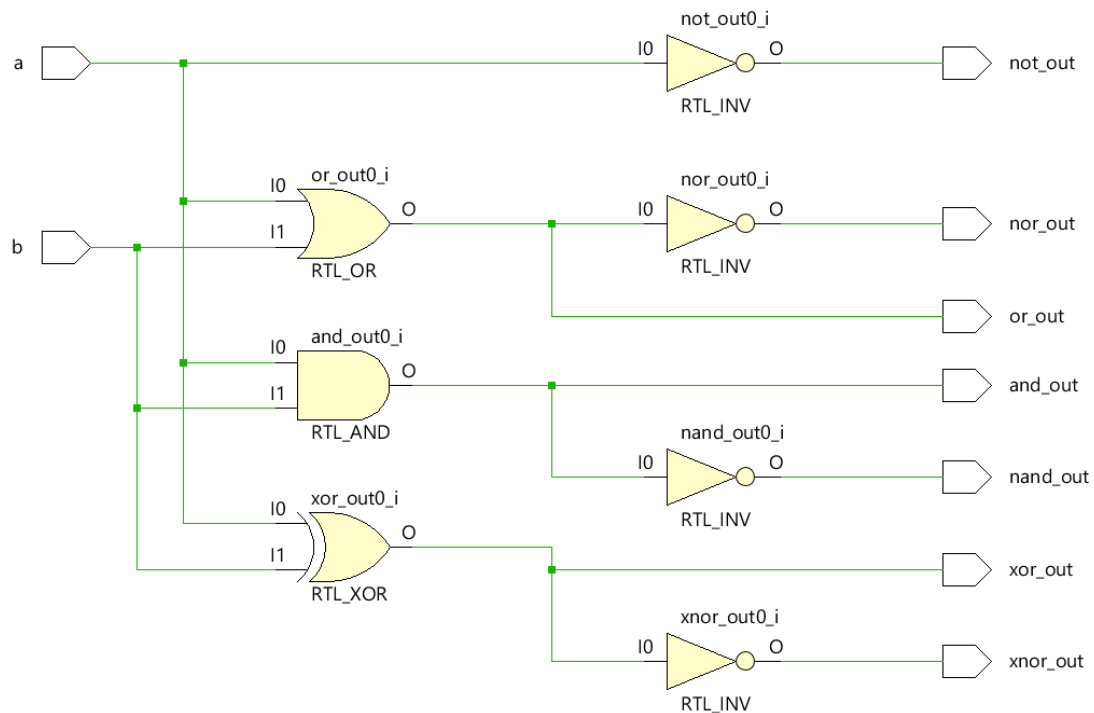
module Basic_Gates_TB;
    reg a, b;
    wire not_out, and_out, nand_out, or_out, nor_out, xor_out, xnor_out;

    Basic_Gates uut
    (.a(a),.b(b),.not_out(not_out),.and_out(and_out),.nand_out(nand_out),.or_out(or_out),
    .nor_out(nor_out),.xor_out(xor_out),.xnor_out(xnor_out));

    initial begin
        a = 0; b = 0;
        #10;
        a = 0; b = 1;
        #10;
        a = 1; b = 0;
        #10;
        a = 1; b = 1; #10;
        $display("%b %b|%b %b %b %b %b %b %b", a, b, not_out, and_out, nand_out,
        or_out, nor_out, xor_out, xnor_out);
        $finish;
    end
endmodule

```





HALF ADDER

```

module Half_adder(input a,b,output reg s,c);
    always @(*) begin
        s = a ^ b;
        c = a & b;
    end
endmodule

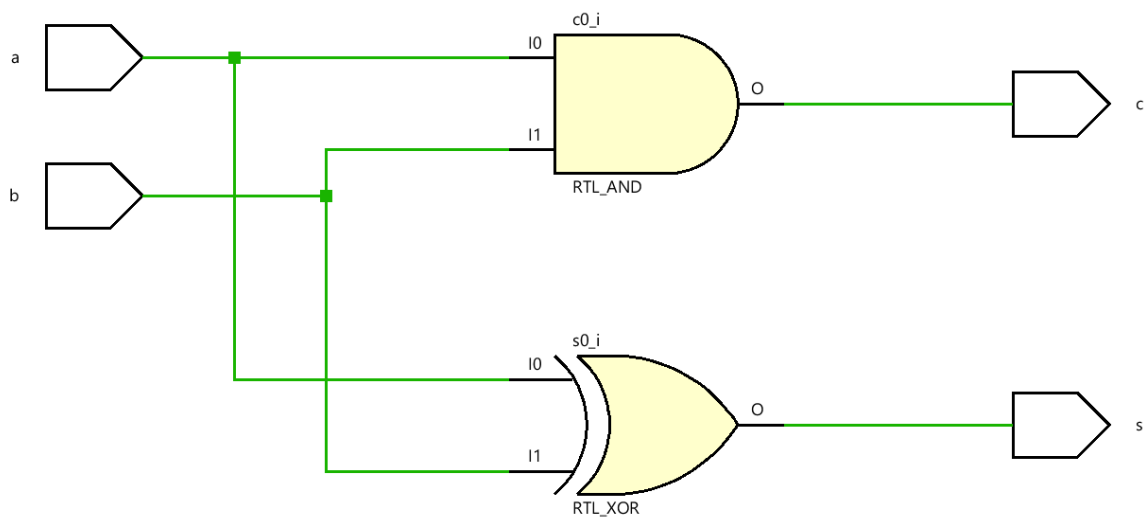
```

```

module Half_adder_TB;
    reg a, b;
    wire s, c;
    Half_Adder uut (.a(a),.b(b),.s(s),.c(c));
    initial begin
        a = 0; b = 0; #10;
        a = 0; b = 1; #10;
        a = 1; b = 0; #10;
        a = 1; b = 1; #10;
        $display("%b %b| %b %b", a, b, s, c);
        $finish;
    end
endmodule

```

Name	Value	0 ns	10 ns	20 ns	30 ns	40 ns
a	1					
b	1					
s	0					
c	1					



FULL ADDER

```

module Full_adder(input a,b,cin,output reg sum,carry);
    always @(*) begin
        sum = a ^ b ^ cin;
        carry = (a & b) | (b & cin) | (cin & a);
    end
endmodule

```

```

module Full_adder_TB;
    reg a, b, cin;
    wire sum, carry;

    Full_Adder uut (.a(a),.b(b),.cin(cin),.sum(sum),.carry(carry));

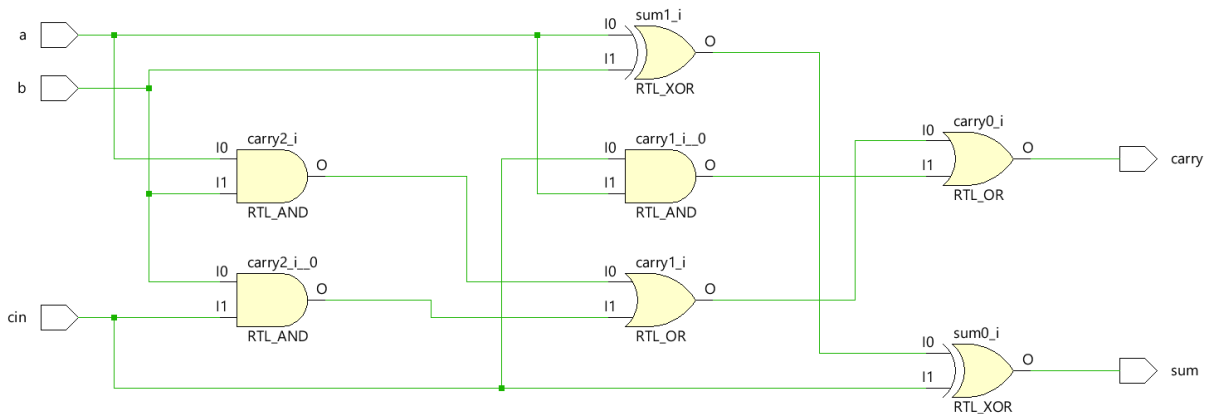
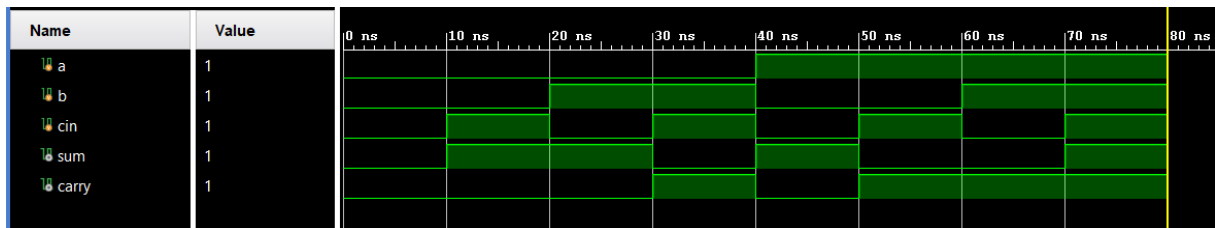
    initial begin
        a = 0; b = 0; cin = 0; #10;
        a = 0; b = 0; cin = 1; #10;
    end
endmodule

```

```

a = 0; b = 1; cin = 0; #10;
a = 0; b = 1; cin = 1; #10;
a = 1; b = 0; cin = 0; #10;
a = 1; b = 0; cin = 1; #10;
a = 1; b = 1; cin = 0; #10;
a = 1; b = 1; cin = 1; #10;
$display("%b %b %b | %b    %b", a, b, cin, sum, carry);
$finish;
end
endmodule

```



HALF SUBTRACTOR

```

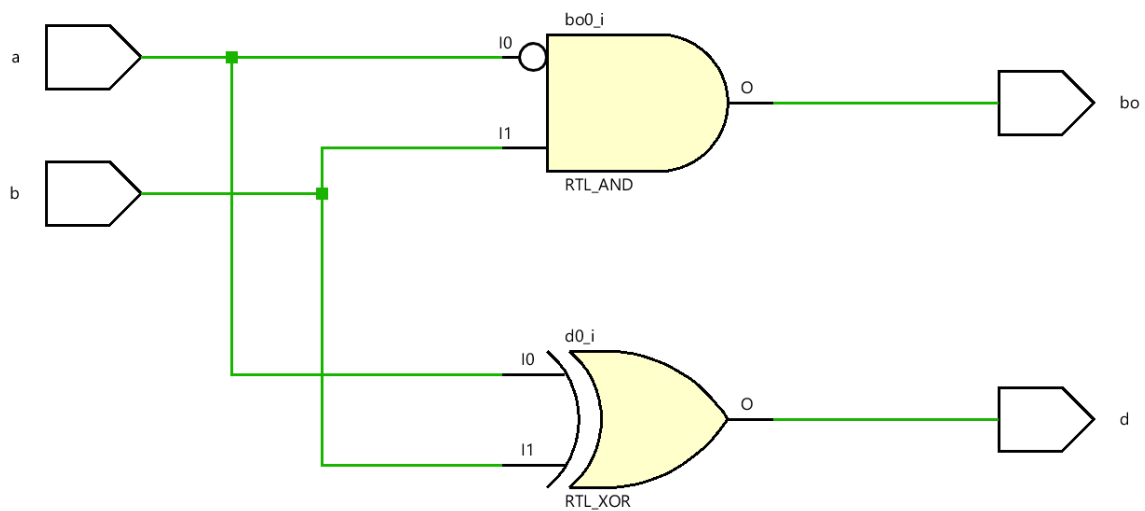
module Half_Subtractor(input a,b,output reg d,bo);
    always @(*) begin
        d =a ^ b;
        bo =~ a & b;
    end
endmodule

```

```

module Half_adder_TB;
  reg a, b;
  wire d, bo;
  Half_Subtractor uut (.a(a),.b(b),.d(d),.bo(bo));
  initial begin
    a = 0; b = 0; #10;
    a = 0; b = 1; #10;
    a = 1; b = 0; #10;
    a = 1; b = 1; #10;
    $display("%b %b|%b %b", a, b, d, bo);
    $finish;
  end
endmodule

```



Name	Value	0 ns	10 ns	20 ns	30 ns	40 ns
a	1					
b	1					
d	0					
bo	0					

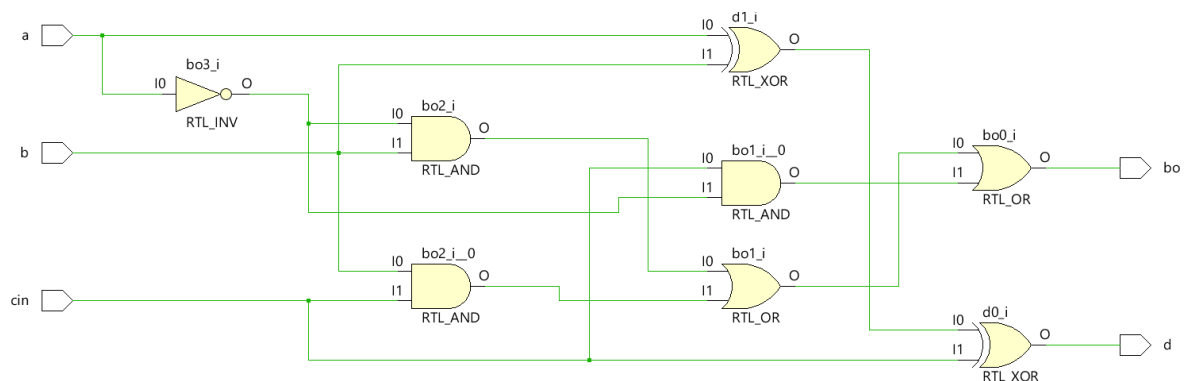
FULL SUBTRACTOR

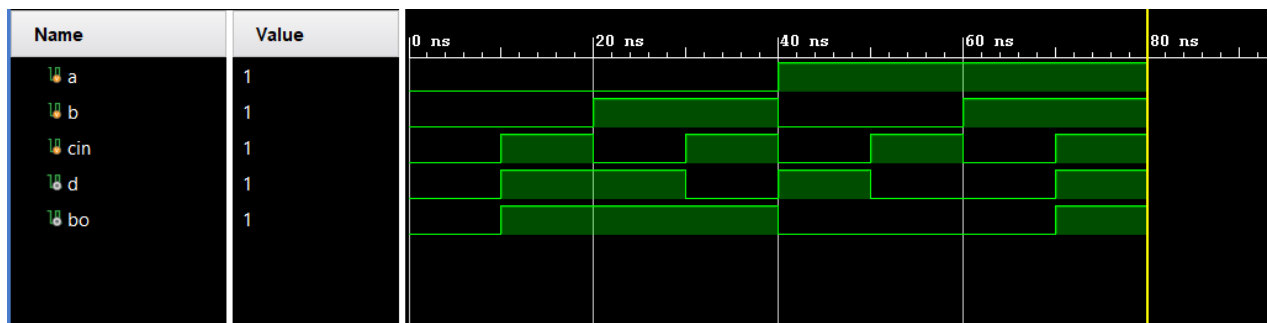
```
module Full_Subtractor(input a,b,cin,output reg d,bo);
    always @(*) begin
        d  = a ^ b ^ cin;
        bo = (~a & b) | (b & cin) | (cin & ~a);
    end
endmodule
```

```
module Full_Subtractor_TB;

    reg a, b, cin;
    wire d,bo;

    Full_Subtractor uut (.a(a),.b(b),.cin(cin),.d(d),.bo(bo));
    initial begin
        a = 0; b = 0; cin = 0; #10;
        a = 0; b = 0; cin = 1; #10;
        a = 0; b = 1; cin = 0; #10;
        a = 0; b = 1; cin = 1; #10;
        a = 1; b = 0; cin = 0; #10;
        a = 1; b = 0; cin = 1; #10;
        a = 1; b = 1; cin = 0; #10;
        a = 1; b = 1; cin = 1; #10;
        $display("%b %b %b | %b    %b", a, b, cin, d, bo);
        $finish;
    end
endmodule
```





2:1 MUX

```
module mux2to1(input a,b,sel,output reg y);

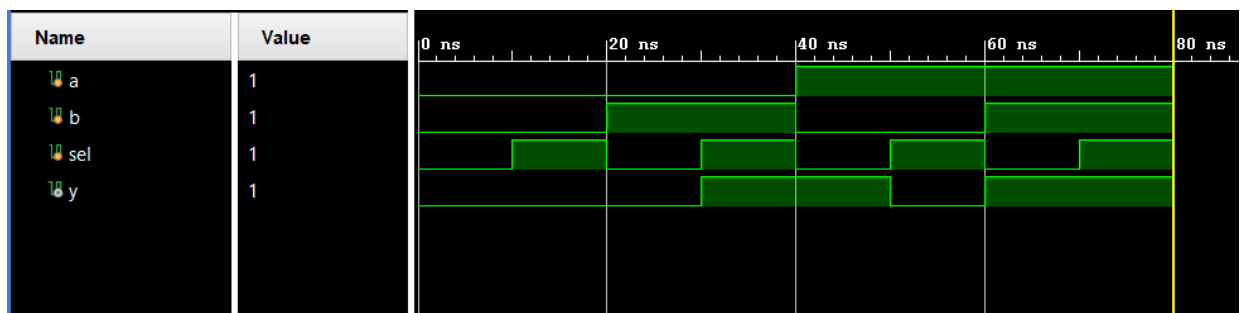
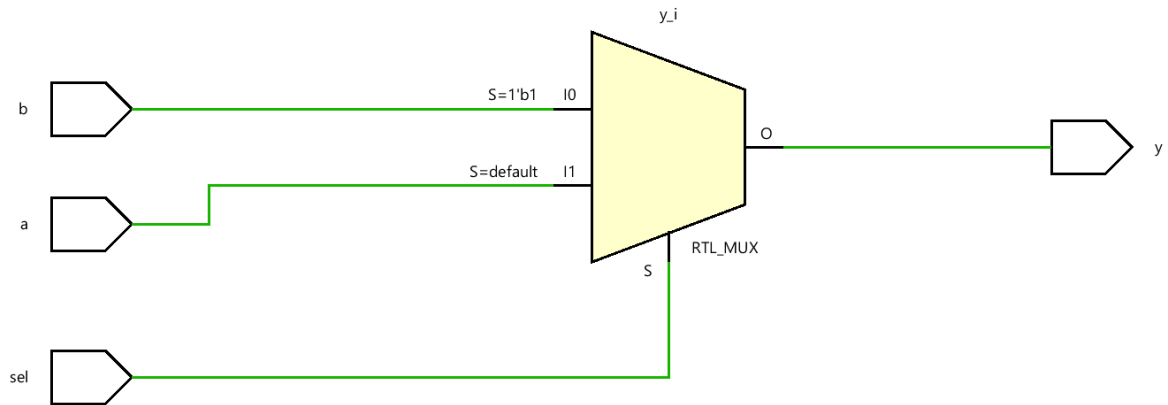
    always @(*) begin
        if (sel)
            y = b;
        else
            y = a;
        end
    endmodule
```

```
module mux2to1_TB;

    reg a, b, sel;
    wire y;

    mux2to1 uut (.a(a),.b(b),.sel(sel),.y(y));

    initial begin
        a = 0; b = 0; sel = 0; #10;
        a = 0; b = 0; sel = 1; #10;
        a = 0; b = 1; sel = 0; #10;
        a = 0; b = 1; sel = 1; #10;
        a = 1; b = 0; sel = 0; #10;
        a = 1; b = 0; sel = 1; #10;
        a = 1; b = 1; sel = 0; #10;
        a = 1; b = 1; sel = 1; #10;
        $display("%b %b %b | %b", a, b, sel, y);
        $finish;
    end
endmodule
```

FOUR BIT COMPARATOR

```

module comparator_4bit(input [3:0]A,[3:0]B,output reg A_LE_B,A_EQ_B,A_GR_B);

always @(*) begin

    if (A<B) begin
        A_LE_B = 1;
        A_EQ_B = 0;
        A_GR_B = 0;
    end

    else if (A==B) begin
        A_LE_B = 0;
        A_EQ_B = 1;
        A_GR_B = 0;
    end

    else begin
        A_LE_B = 0;
        A_EQ_B = 0;
    end
end

```

```

A_GR_B = 1;
end

end
endmodule

```

```

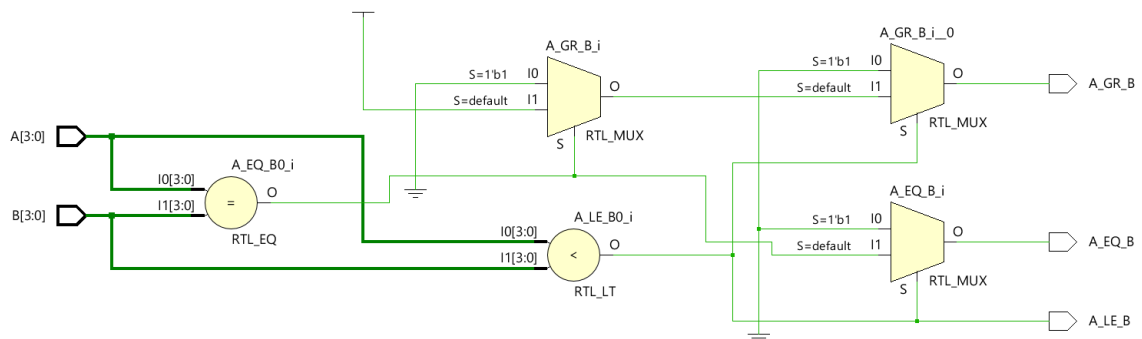
module comparator_4bit_TB;
reg [3:0]A,B ;
wire A_LE_B,A_EQ_B,A_GR_B;

comparator_4bit uut
(.A(A),.B(B),.A_LE_B(A_LE_B),.A_EQ_B(A_EQ_B),.A_GR_B(A_GR_B));

initial begin
A=4'b0000;B=4'b1110; #10
A=4'b1111;B=4'b0001; #10
A=4'b0010;B=4'b0010; #10
A=4'b1101;B=4'b0011; #10
A=4'b0100;B=4'b1010; #10
A=4'b1011;B=4'b0101; #10
A=4'b0110;B=4'b0110; #10
A=4'b1001;B=4'b0111; #10

$finish;
$display("Time=%0t,A=%b,B=%b,A_LE_B=%b,A_EQ_B=%b,A_GR_B=%b",$time,A,B,A_LE_B,A_
EQ_B,A_GR_B);
end
endmodule

```





4-BIT RIPPLE CARRY ADDER

```

module half_adder(input A,B,output reg S,C);
always @(*) begin
S<=A^B;
C<=A&B;
end
endmodule

module Full_adder(input A,B,C,output Sum ,Carry);
wire S1,C1,C2;
half_adder H1(.A(A),.B(B),.S(S1),.C(C1));
half_adder H2(.A(S1),.B(C),.S(Sum),.C(C2));
assign Carry= C1|C2;
endmodule

module Ripple_carry_adder(input [3:0]A,[3:0]B,Cin,output [3:0]S,Cout);
wire [2:0]C;

Full_adder FA1(.A(A[0]),.B(B[0]),.C(Cin),.Sum(S[0]),.Carry(C[0]));
Full_adder FA2(.A(A[1]),.B(B[1]),.C(C[0]),.Sum(S[1]),.Carry(C[1]));
Full_adder FA3(.A(A[2]),.B(B[2]),.C(C[1]),.Sum(S[2]),.Carry(C[2]));
Full_adder FA4(.A(A[3]),.B(B[3]),.C(C[2]),.Sum(S[3]),.Carry(Cout));

endmodule

```

```

module Ripple_carry_adder_TB;
reg [3:0]A;
reg [3:0]B;
reg Cin;
wire [3:0]S;
wire Cout;

Ripple_carry_adder uut (.A(A),.B(B),.Cin(Cin),.S(S),.Cout(Cout));

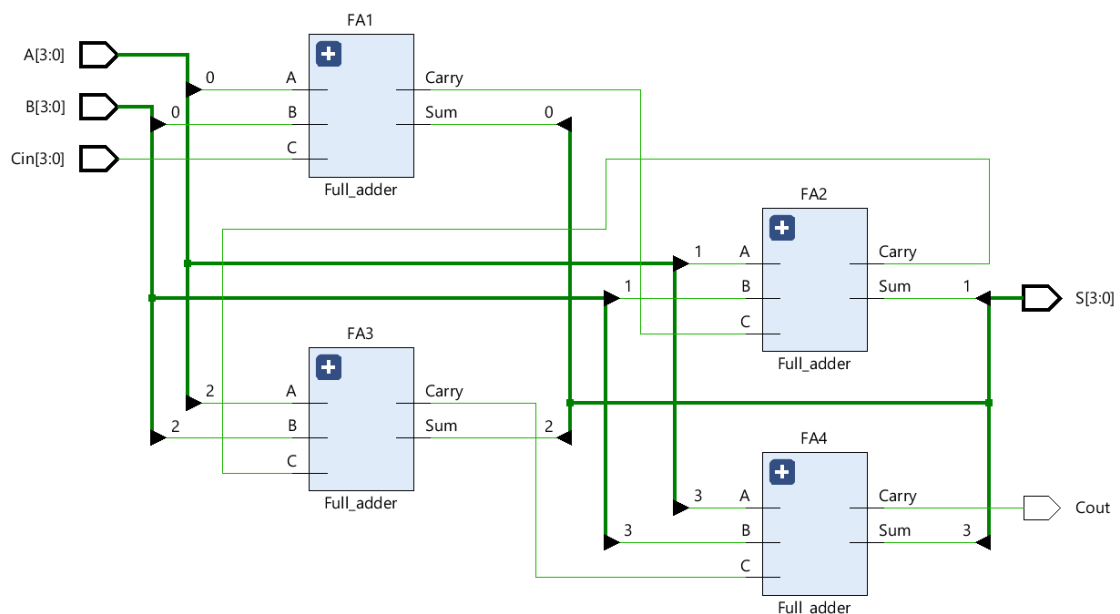
initial begin

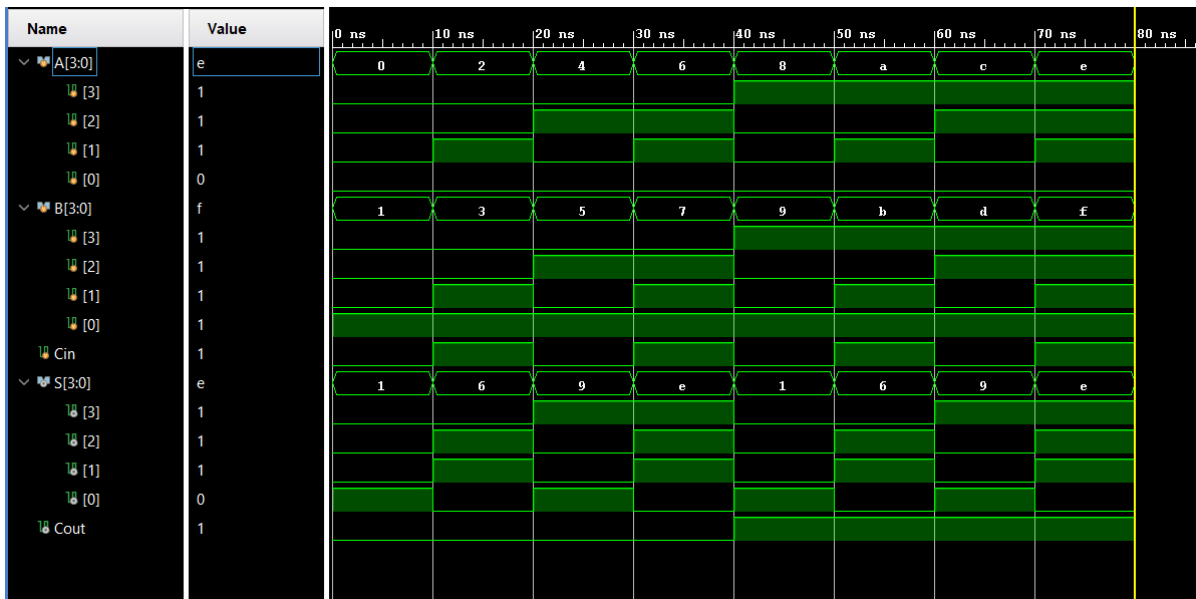
A=4'b0000; B=4'b0001; Cin=0; // 0+1+0=1
#10 A=4'b0010; B=4'b0011; Cin=1;// 2+3+1=6
#10 A=4'b0100; B=4'b0101; Cin=0;// 4+5+0=9
#10 A=4'b0110; B=4'b0111; Cin=1;// 6+7+1=14
#10 A=4'b1000; B=4'b1001; Cin=0;// 8+9+0=17
#10 A=4'b1010; B=4'b1011; Cin=1;// 10+11+1=22
#10 A=4'b1100; B=4'b1101; Cin=0;// 12+13+0=25
#10 A=4'b1110; B=4'b1111; Cin=1;// 14+15+1=30

end
initial begin
$display ("Time=%0t,A=%b,B=%b,Cin=%b,S=%t,Cout=%t", $time,A,B,Cin,S,Cout);
end

endmodule

```





LATCHES

JK Latch

```

module JK_latch (J,K,enable,reset,Q,Qn);
    input J,K,enable,reset;
    output reg Q, Qn;

    always @(*) begin
        if (reset) begin
            Q = 0;
            Qn = 1;
        end
        else if (enable) begin
            if (J == 0 && K == 0) begin
                Q = Q;
                Qn = Qn;
            end
            else if (J == 0 && K == 1) begin
                Q = 0;
                Qn = 1;
            end
            else if (J == 1 && K == 0) begin
                Q = 1;
                Qn = 0;
            end
            else begin
                Q = 1'bx;
                Qn = 1'bx;
            end
        end
    end
end

```

```

        else begin
            Q = Q;
            Qn = Qn;
        end
    end
endmodule

```

```

module JK_latch_TB;
    reg J,K,enable,reset;
    wire Q, Qn;

    JK_latch uut (.J(J),.K(K),.enable(enable),.reset(reset),.Q(Q),.Qn(Qn));

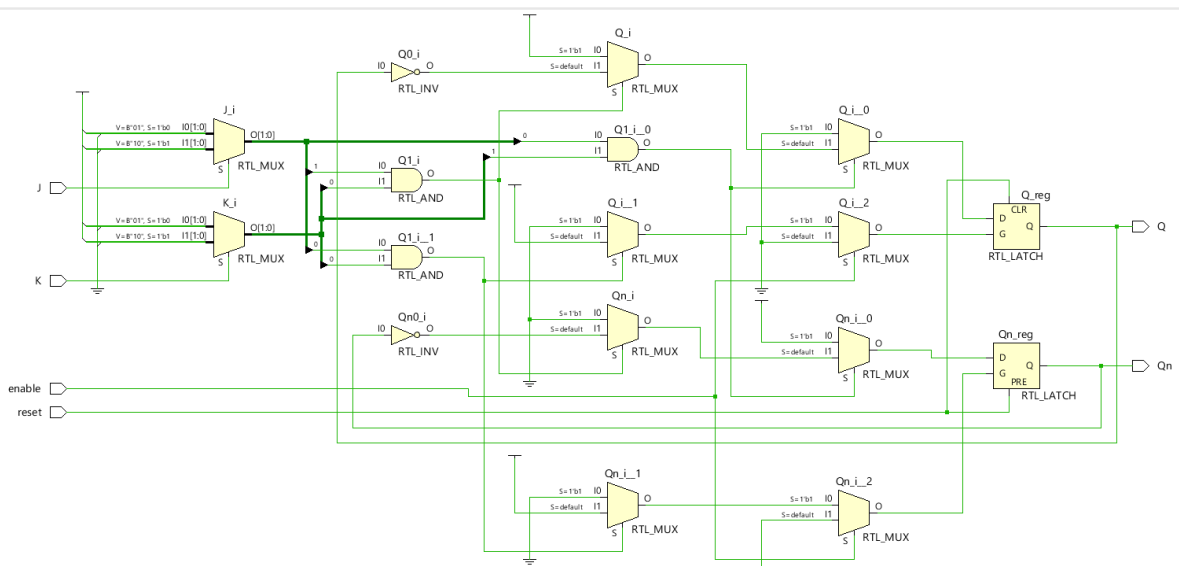
    initial begin
        $monitor("Time=%0t,reset=%b,enable=%b,J=%b,K=%b,Q=%b,Qn=%b", $time, reset,
        enable, J, K, Q, Qn);

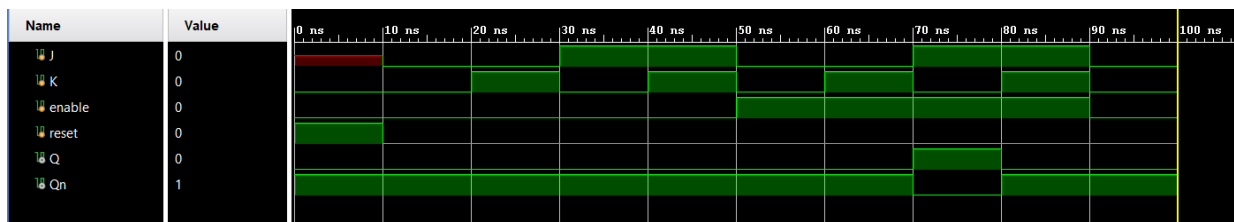
        reset = 1; enable = 0; K = J; K = 0; #10;
        reset = 0;

        enable = 0; J = 0; K = 0; #10;
        enable = 0; J = 0; K = 1; #10;
        enable = 0; J = 1; K = 0; #10;
        enable = 0; J = 1; K = 1; #10;
        enable = 1; J = 0; K = 0; #10;
        enable = 1; J = 0; K = 1; #10;
        enable = 1; J = 1; K = 0; #10;
        enable = 1; J = 1; K = 1; #10;
        enable = 0; J = 0; K = 0; #10;

        $finish;
    end
endmodule

```





SR Latch

```

module SR_latch (S,R,enable,reset,Q,Qn);
    input S,R,enable,reset;
    output reg Q, Qn;

    always @(*) begin
        if (reset) begin
            Q = 0;
            Qn = 1;
        end
        else if (enable) begin

            if (S == 0 && R == 0) begin
                Q = Q;
                Qn = Qn;
            end
            else if (S == 0 && R == 1) begin
                Q = 0;
                Qn = 1;
            end
            else if (S == 1 && R == 0) begin
                Q = 1;
                Qn = 0;
            end
            else begin
                Q = 1'bx;
                Qn = 1'bx;
            end
        end
        else begin
            Q = Q;
            Qn = Qn;
        end
    end
endmodule

```

```

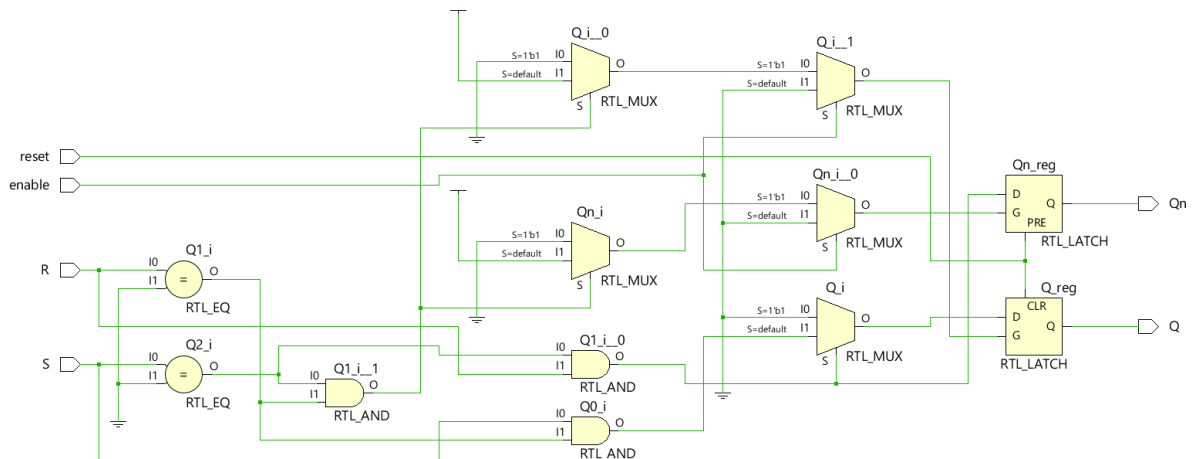
module SR_latch_TB;
    reg S,R,enable,reset;
    wire Q, Qn;

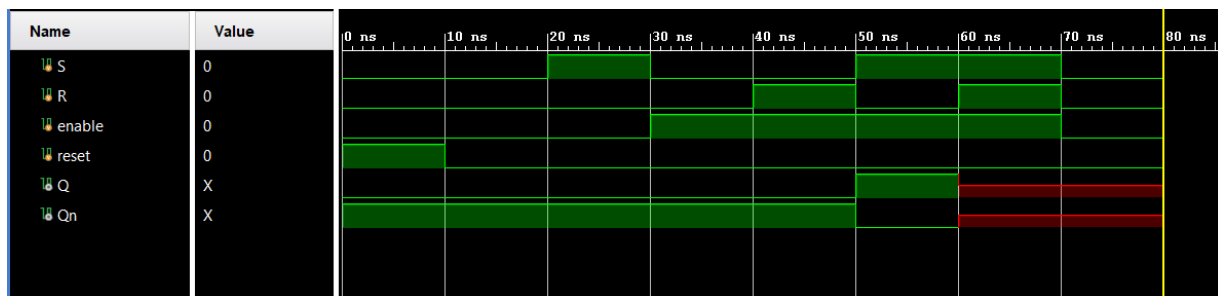
    SR_latch uut (.S(S),.R(R),.enable(enable),.reset(reset),.Q(Q),.Qn(Qn));

    initial begin
        reset = 1; enable = 0; S = 0; R = 0;
        #10;
        reset = 0;

        enable = 0; S = 0; R = 0; #10;
        enable = 0; S = 1; R = 0; #10;
        enable = 1; S = 0; R = 0; #10;
        enable = 1; S = 0; R = 1; #10;
        enable = 1; S = 1; R = 0; #10;
        enable = 1; S = 1; R = 1; #10;
        enable = 0; S = 0; R = 0; #10;
        $monitor("Time=%t ,reset=%b,enable=%b,S=%b R=%b,Q=%b Qn=%b", $time, reset,
        enable, S, R, Q, Qn);
        $finish;
    end
endmodule

```





D Latch

```
module D_latch (input D,enable,reset,output reg Q,output Q_bar);

always @(*) begin
    if (reset)
        Q = 1'b0;
    else if (enable)
        Q = D;
end
assign Q_bar=~Q;
endmodule
```

```
module D_latch_TB;
    reg D;
    reg enable;
    reg reset;
    wire Q,Q_bar;
    D_latch uut (.D(D),.enable(enable),.reset(reset),.Q(Q),.Q_bar(Q_bar));

    initial begin

        D = 0; enable = 0; reset = 0;

        #5 reset = 1;
        #10 reset = 0;

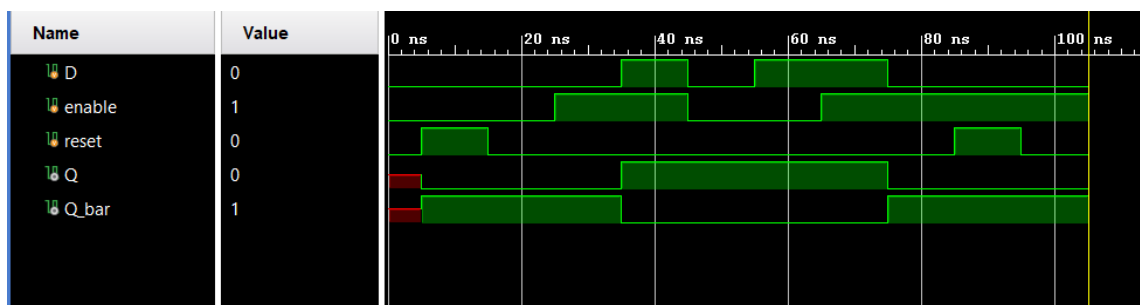
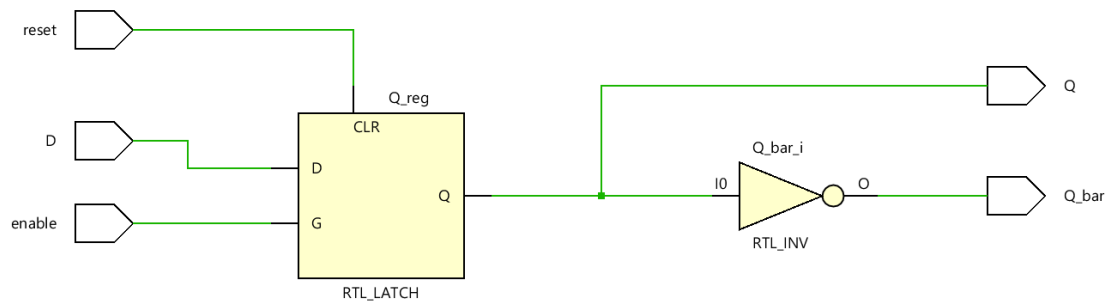
        #10 enable = 1; D = 0;
        #10 D = 1;
        #10 enable = 0; D = 0;
        #10 D = 1;
        #10 enable = 1;
        #10 D = 0;
        #10 reset = 1;
        #10 reset = 0;
    end
endmodule
```

```

        #10 $finish;
    end

endmodule

```



T Latch

```

module T_latch (input T,enable,reset,output reg Q,output Q_bar);

always @(*) begin
    if (reset)
        Q = 1'b0;
    else if (enable) begin
        if (T)
            Q = ~Q;
        else
            Q = Q;
    end
    else
        Q = Q;
end
assign Q_bar=~Q;

```

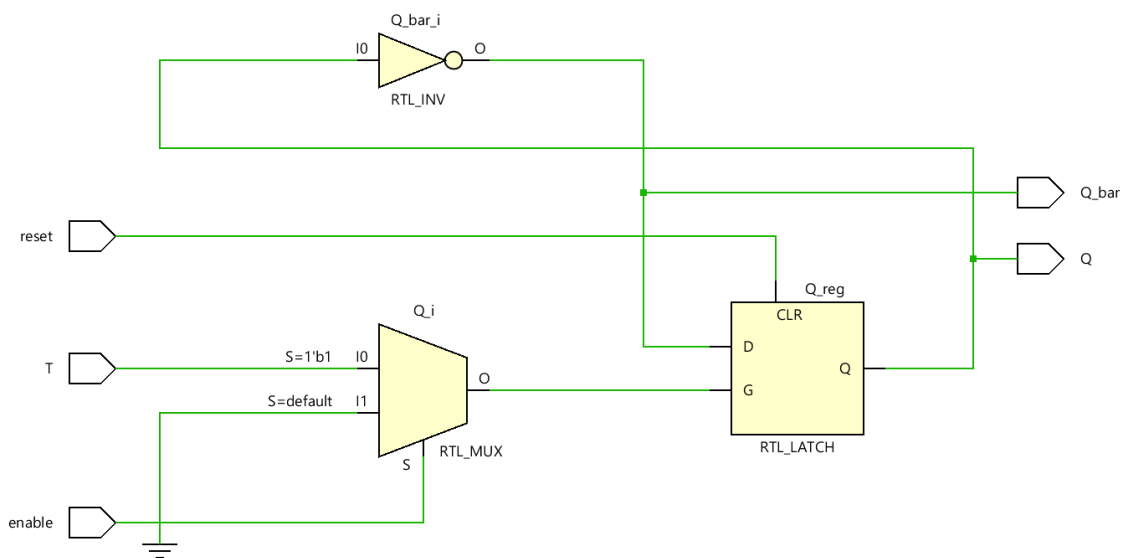
```
endmodule
```

```
module T_latch_TB;
    reg T;
    reg enable;
    reg reset;
    wire Q,Q_bar;
    T_latch uut (.T(T),.enable(enable),.reset(reset),.Q(Q),.Q_bar(Q_bar));

    initial begin

        T = 0; enable = 0; reset = 1;

        #10 reset = 0;
        #10 T = 1;
        #10 T = 0; enable = 1;
        #10 T = 1;
        #20 T = 0;
        #10 T = 1;
        #10 reset = 1;
        #10 reset = 0;
        #10 $finish;
    end
endmodule
```



Name	Value	0 ns	10 ns	20 ns	30 ns	40 ns	50 ns	60 ns	70 ns	80 ns	90 ns	100 ns
T	1											
enable	1											
reset	0											
Q	1											
Q_bar	0											

FLIPFLOPS

SR Flipflop

```

module SR_FF( input clk,reset,S,R,output reg q, output q_bar);

always @(posedge clk)begin
if (!reset) q<=0;
else begin
case({S,R})
2'b00 : q<=q;
2'b01 : q<=0;
2'b10 : q<=1;
2'b11 : q<=1'bx;
endcase
end
end
assign q_bar=~q;
endmodule

```

```

module SR_FF_TB;
reg clk;
reg reset;
reg S;
reg R;
wire q;
wire q_bar;

SR_FF uut(.clk(clk),.reset(reset),.S(S),.R(R),.q(q),.q_bar(q_bar));

initial begin
clk=0 ;
forever #5 clk=~clk;
end
initial begin

reset= 0;S=0;R=0;
#10 reset=1;

```

```
S=0;R=0;
```

```
#10
```

```
S=0;R=1;
```

```
#10
```

```
S=1;R=0;
```

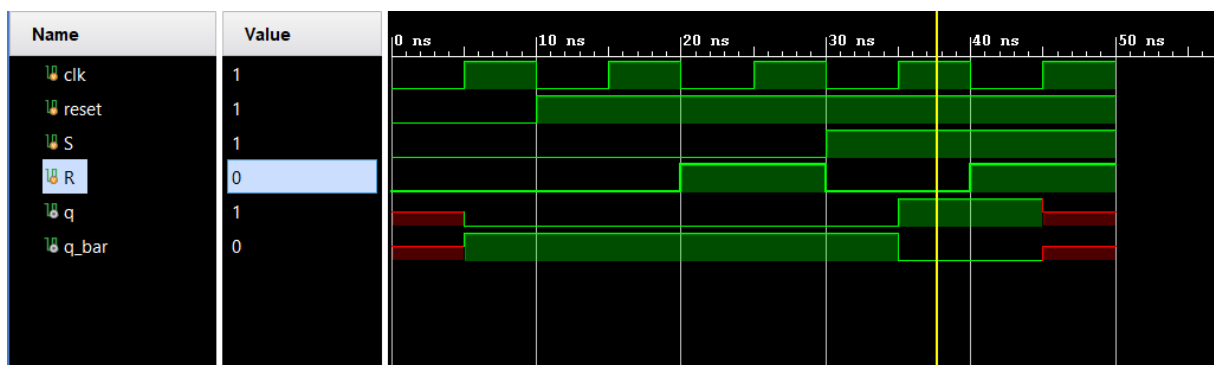
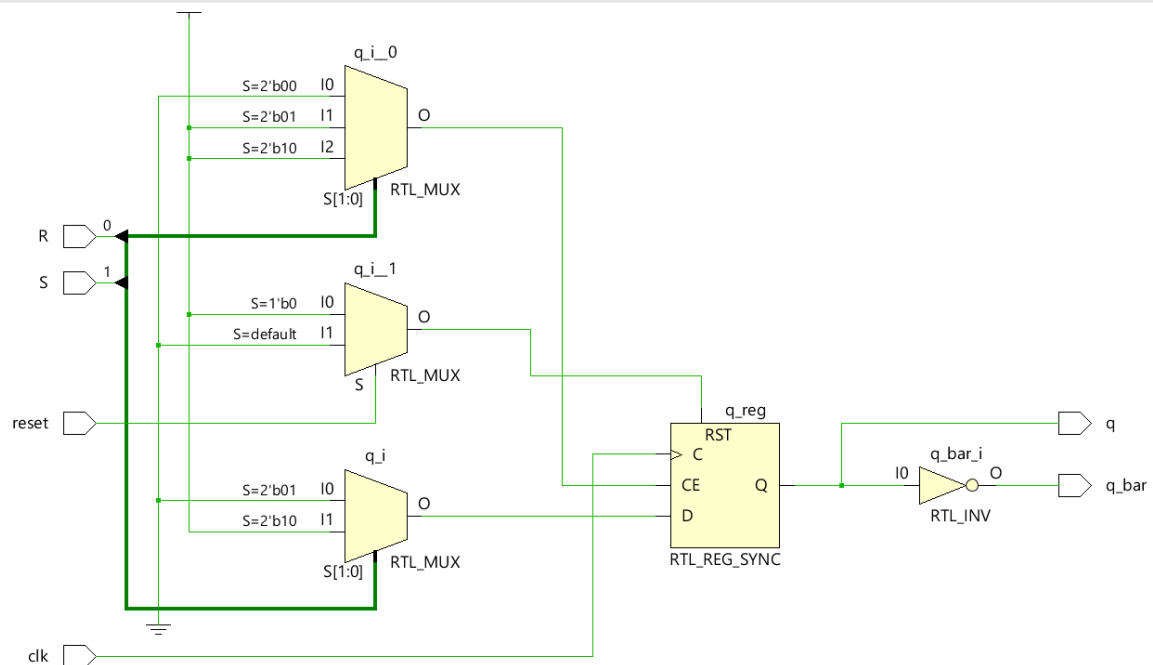
```
#10
```

```
S=1;R=1;
```

```
$display("Time=%0t,clk=%b,reset=%b,S=%b,R=%b,q=%b,q_bar=%b",$time,clk,reset,S,R,q,q_bar);
```

```
end
```

```
endmodule
```



D Flipflop

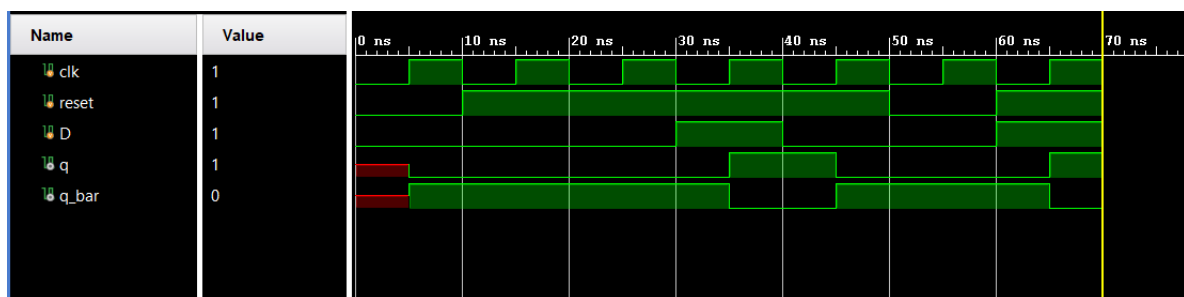
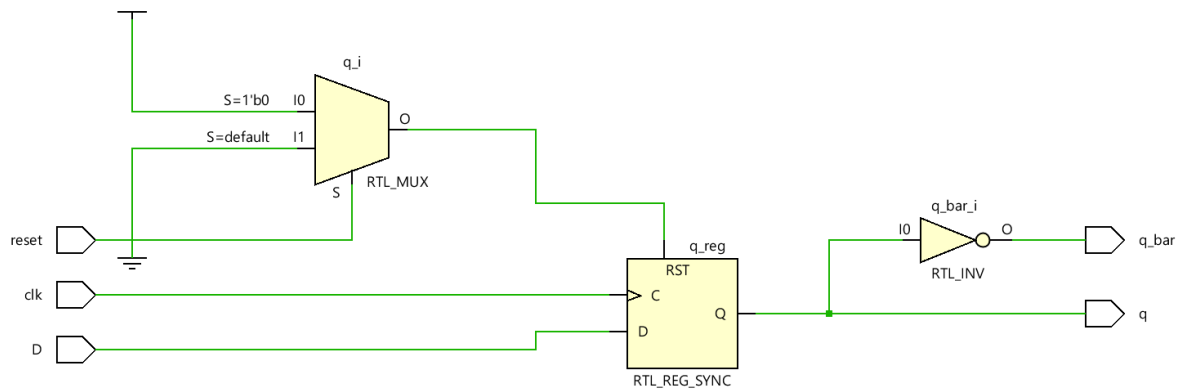
```
module D_FF(input clk,reset,D,output reg q,output q_bar);
    always @(posedge clk) begin
        if (!reset)
            q<=1'b0;
        else
            q<=D;
        end
    assign q_bar=~q;
endmodule
```

```
module D_FF_TB;
    reg clk,reset,D;
    wire q,q_bar;

    D_FF uut (.clk(clk),.reset(reset),.D(D),.q(q),.q_bar(q_bar));

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    initial begin
        reset = 0; D = 0;
        #10 reset = 1;
        #10 D = 0;
        #10 D = 1;
        #10 D = 0;
        #10 reset = 0;
        #10 reset = 1;
        D = 1;
        $display("Time=%0t,reset=%b,D=%b,q=%b,q_bar=%b",$time,reset,D,q,q_bar);
        #10 $finish;
    end
endmodule
```



JK Flipflop

```

module JK_FF(
  input clk,
  input reset,
  input J,
  input K,
  output reg q,
  output q_bar);

  always @(posedge clk) begin
    if (!reset)
      q<=0;
    else begin
      case({J, K})
        2'b00: q <= q;
        2'b01: q <= 1'b0;
        2'b10: q <= 1'b1;
        2'b11: q <= ~q;
      endcase
    end
  end
  assign q_bar = ~q;
endmodule

```

```

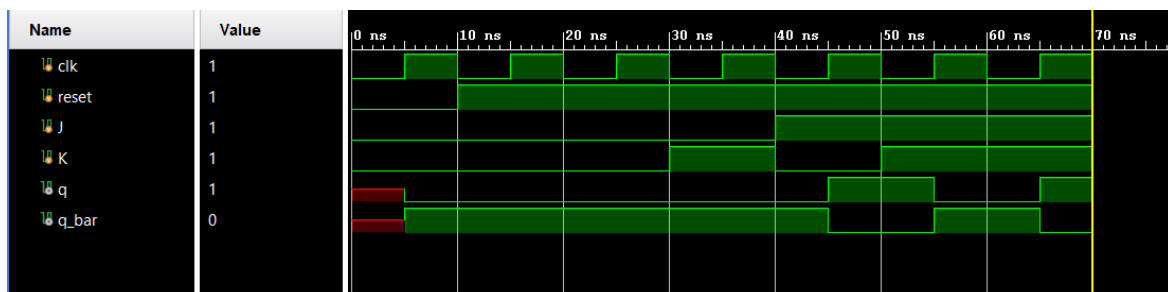
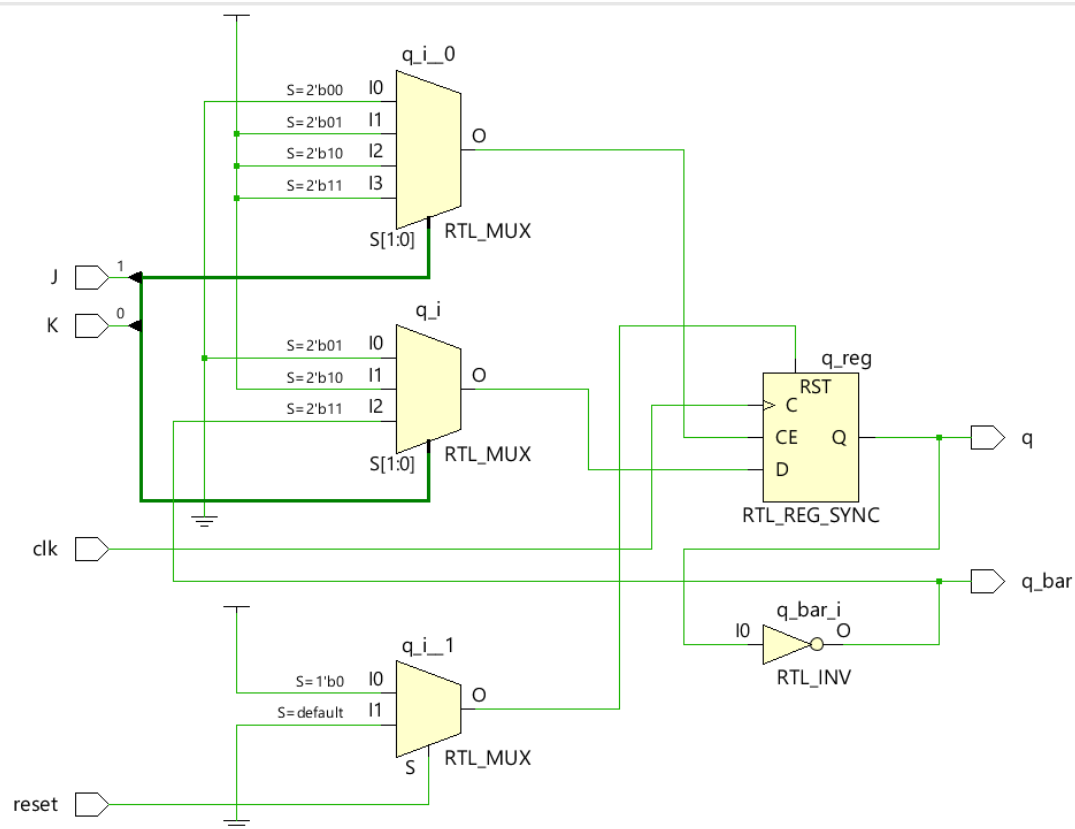
module JK_FF_TB;
reg clk;
reg reset;
reg J;
reg K;
wire q;
wire q_bar;

JK_FF uut (.clk(clk),.reset(reset),.J(J),.K(K),.q(q),.q_bar(q_bar));

    initial begin
$display("Time=%0t reset=%b J=%b K=%b q=%b q_bar=%b", $time, reset, J,K, q,
q_bar);
        clk = 0;
        forever #5 clk = ~clk;
    end
    initial begin
        reset= 0; J = 0; K = 0;
        #10 reset = 1;

        #10 J = 0; K = 0;
        #10 J = 0; K = 1;
        #10 J = 1; K = 0;
        #10 J = 1; K = 1;
        #10 $finish;
    end
end
endmodule

```

T Flipflop

```

module T_FF(input clk,reset,T,output reg q,output q_bar);

    always @(posedge clk) begin
        if (!reset)
            q<=1'b0;
        else if (T)
            q<=~q;
        else
    
```

```

        q<=q;
    end
    assign q_bar = ~q;
endmodule

```

```

module T_FF_tb;
    reg clk;
    reg reset;
    reg T;
    wire q;
    wire q_bar;

    T_FF uut (.clk(clk),.reset(reset),.T(T),.q(q),.q_bar(q_bar));
    initial clk = 0;
    always #5 clk = ~clk;

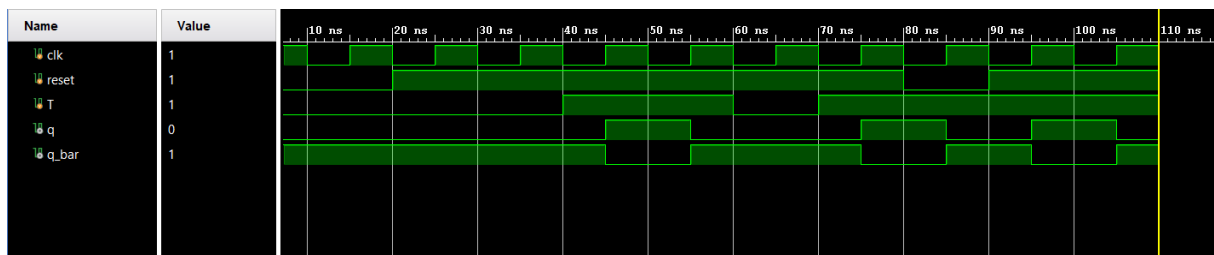
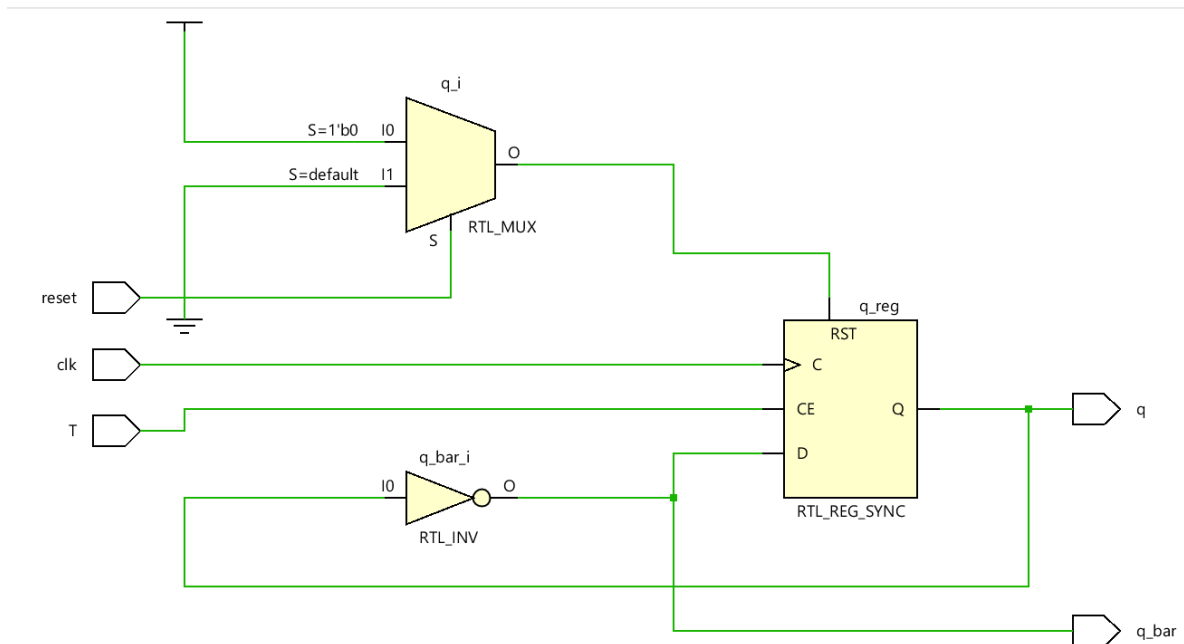
    initial begin
        reset = 0; T = 0;
        #10;

        reset = 0;
        #10;
        reset = 1;

        #10; T = 0;
        #10; T = 1;
        #10; T = 1;
        #10; T = 0;
        #10; T = 1;

        #10; reset = 0;
        #10; reset = 1;
        #20;
        $finish;
    end
    initial begin
        $monitor("Time=%0t,reset=%b, T=%b, q=%b, q_bar=%b", $time, reset, T, q, q_bar);
    end
endmodule

```



SHIFT REGISTERS

SISO

```

module SISO_behav(input clk,reset,D,output reg Q);

    reg [3:0]temp;

    always @ (posedge clk)begin
        if (reset)
            begin
                temp =4'b0000;
                Q<=0;
            end
        else

```

```

begin
Q<=temp[0];
temp<=temp>>1;
temp[3]<=D;
end
end
endmodule

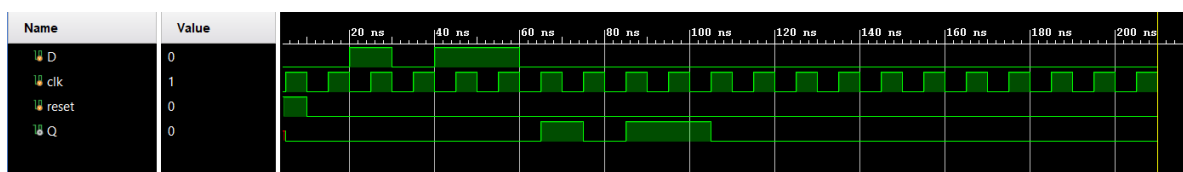
```

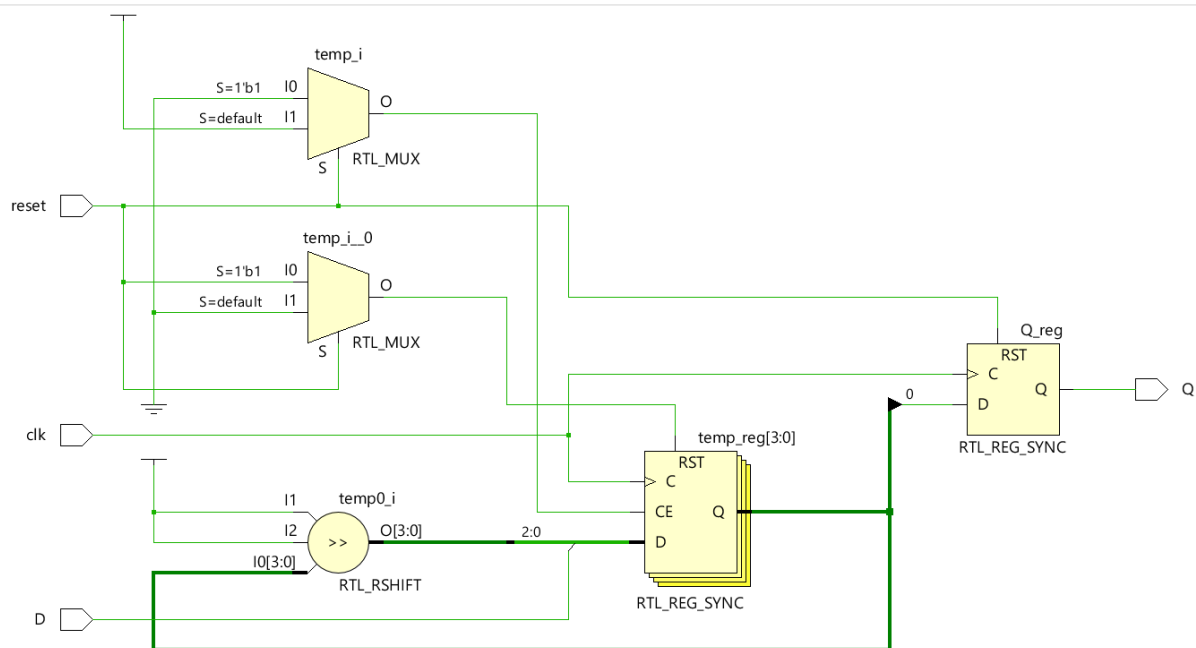
```

module SISO_TB_beha;
reg D,clk,reset;
wire Q;
SISO_beha uut (.D(D),.clk(clk),.reset(reset),.Q(Q));

always #5 clk=~clk;
initial begin
clk=0;
reset=1; D=0;
#10 reset =0;
#10 D =1;
#10 D=0;
#10 D=1;
#10 D =1;
#10 D=0;
#30 D=0;
#120 $finish;
end
endmodule

```





SIPO

```

module SIPO(input clk,reset,D,output reg[3:0] Q);

always @ (posedge clk)begin
if (reset)
begin
Q<=0;
end
else
begin
Q[3]<=D;
Q[2]<=Q[3];
Q[1]<=Q[2];
Q[0]<=Q[1];
end
end
endmodule

```

```

module SIPO_TB_beha;
reg D,clk,reset;
wire [3:0]Q;
SIPO uut (.D(D),.clk(clk),.reset(reset),.Q(Q));

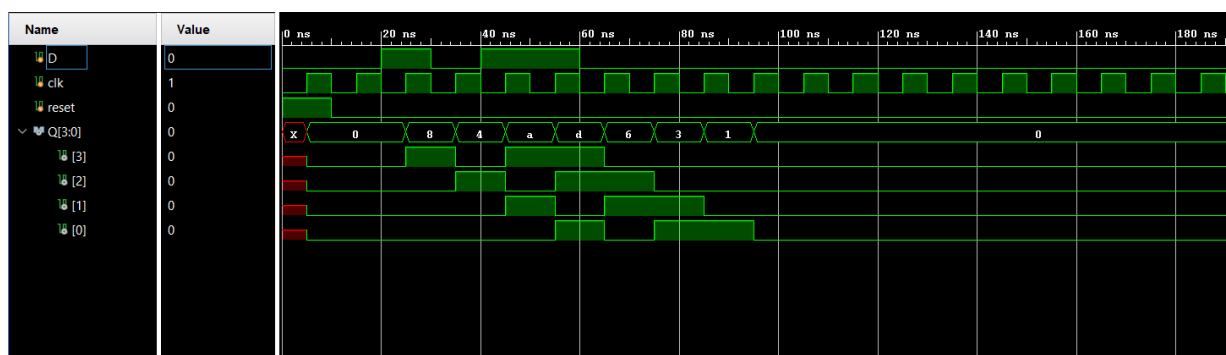
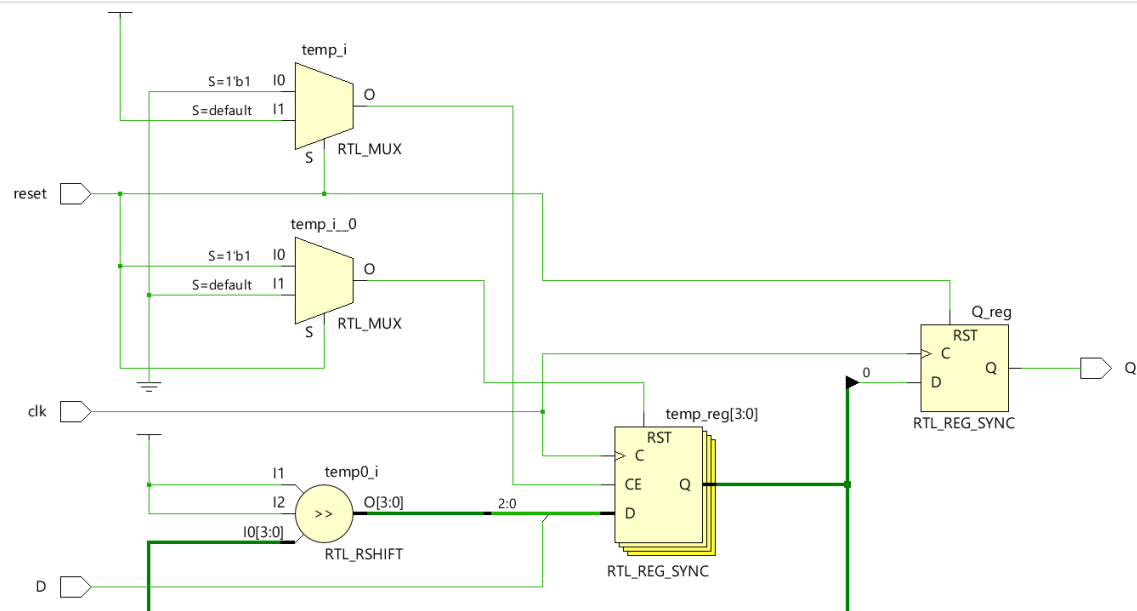
always #5 clk=~clk;

```

```

initial begin
  clk=0;
  reset=1; D=0;
  #10 reset =0;
  #10 D =1;
  #10 D=0;
  #10 D=1;
  #10 D =1;
  #10 D=0;
  #30 D=0;
  #120 $finish;
end
endmodule

```



PIPO

```
module PIPO(input clk,reset,load,[3:0] parallel_in,output reg [3:0]
parallel_out);

    always @(posedge clk or posedge reset) begin
        if (reset)
            parallel_out <= 4'b0000;
        else if (load)
            parallel_out <= parallel_in;
    end
endmodule
```

```
module PIPO_TB;

    reg clk,reset,load, [3:0] parallel_in;
    wire [3:0] parallel_out;

    PIPO uut
    (.clk(clk),.reset(reset),.load(load),.parallel_in(parallel_in),.parallel_out(pa
rallel_out));

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end
    initial begin
        $display("Time=%0t,reset=%b,load=%b,parallel_in=%b,parallel_out=%b", $time,
reset, load, parallel_in, parallel_out);

        reset = 1; load = 0; parallel_in = 4'b0000; #10;
        reset = 0; #10;

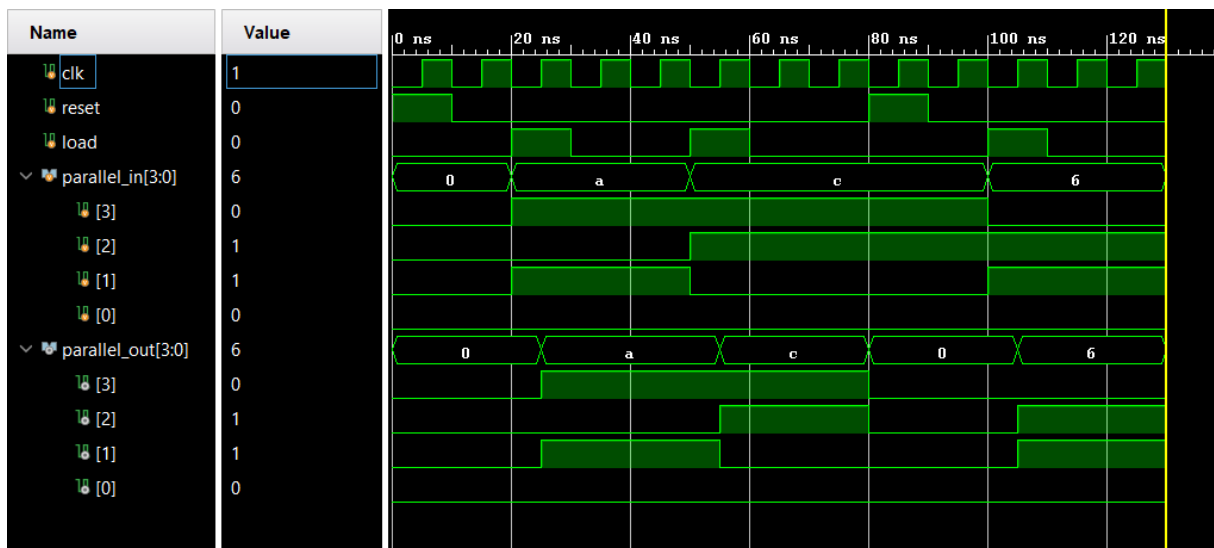
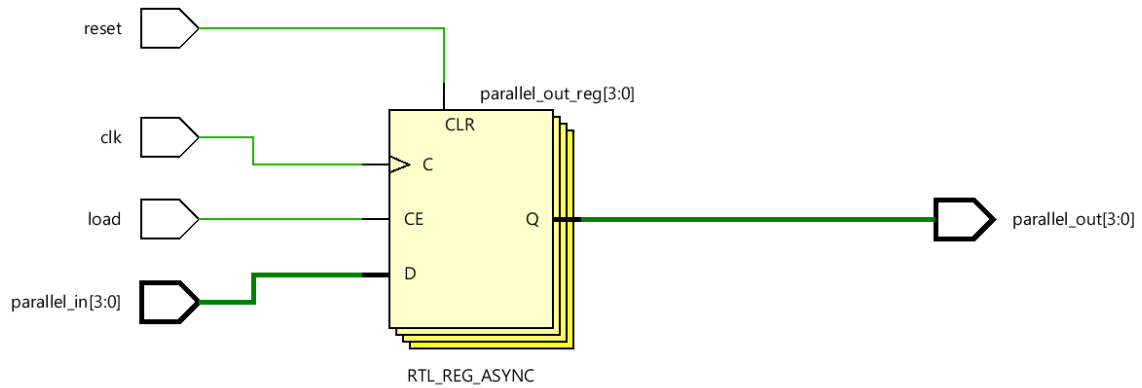
        load = 1; parallel_in = 4'b1010; #10;
        load = 0; #20;

        load = 1; parallel_in = 4'b1100; #10;
        load = 0; #20;

        reset = 1; #10;
        reset = 0; #10;

        load = 1; parallel_in = 4'b0110; #10;
        load = 0; #20;

        $finish;
    end
endmodule
```



PISO

```

module PISO(input clk,reset,load,[3:0] parallel_in,output reg serial_out);

    reg [3:0] shif t_reg;

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            shif t_reg <= 4'b0000;
            serial_out <= 1'b0;
        end else if (load) begin
            shif t_reg <= parallel_in;
            serial_out <= parallel_in[3];
        end else begin
            shif t_reg <= {shif t_reg[2:0], 1'b0};
        end
    end

```



```

        serial_out <= shif t_reg[2];
    end
end
endmodule

```

```

module PISO_TB;

    reg clk,reset,load,[3:0] parallel_in;
    wire serial_out;

    PISO uut
    (.clk(clk),.reset(reset),.load(load),.parallel_in(parallel_in),.serial_out(seri
al_out));

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end
    initial begin

        reset = 1; load = 0; parallel_in = 4'b0000; #10;
        reset = 0; #10;

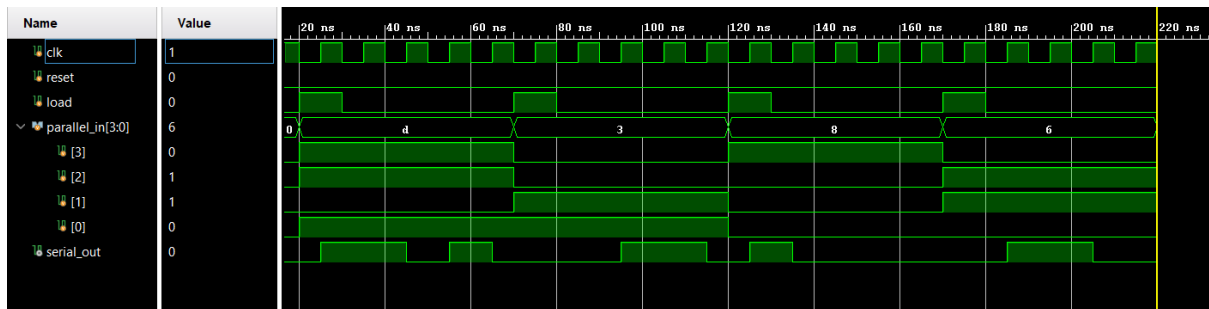
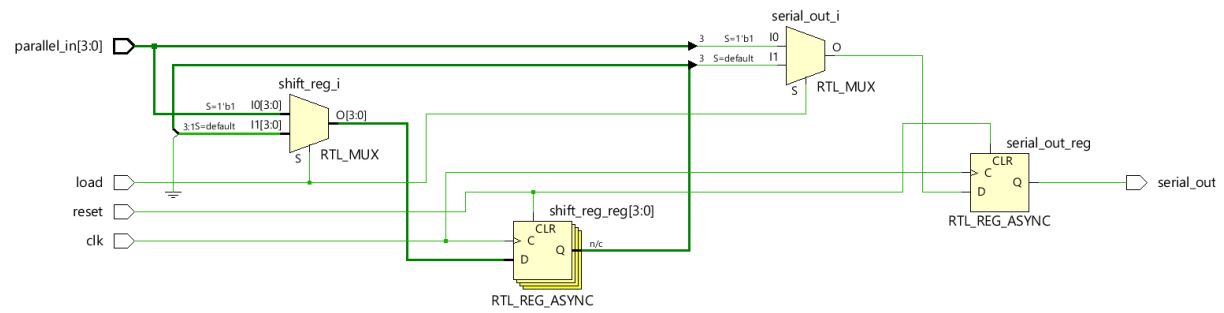
        load = 1; parallel_in = 4'b1101; #10; load = 0;
        #10; #10; #10; #10;

        load = 1; parallel_in = 4'b0011; #10; load = 0;
        #10; #10; #10; #10;

        load = 1; parallel_in = 4'b1000; #10; load = 0;
        #10; #10; #10; #10;

        load = 1; parallel_in = 4'b0110; #10; load = 0;
        #10; #10; #10; #10;
        $display
    ("Time=%0t,reset=%b,parallel_in=%b,serial_out=%b,load=%b",$time,reset,load,para
llep_in,serial_out);
        $finish;
    end
endmodule

```



COUNTERS

4-BIT UP COUNTER

```

module fourbit_upcounter(
    input clk,
    input reset,
    output reg [3:0] count);

    initial count = 4'b0000;

    always @(posedge clk) begin
        if (reset == 1'b1)
            count <= 4'b0000;
        else
            count <= count + 1;
        end
    endmodule

```

```

module fourbit_upcounter_TB;
    reg clk, reset;
    wire [3:0] count;

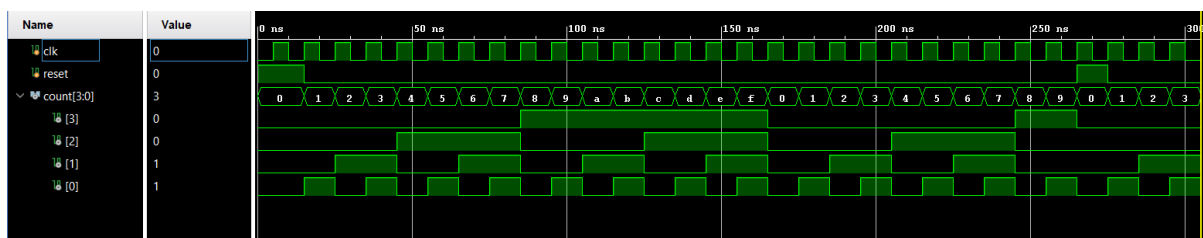
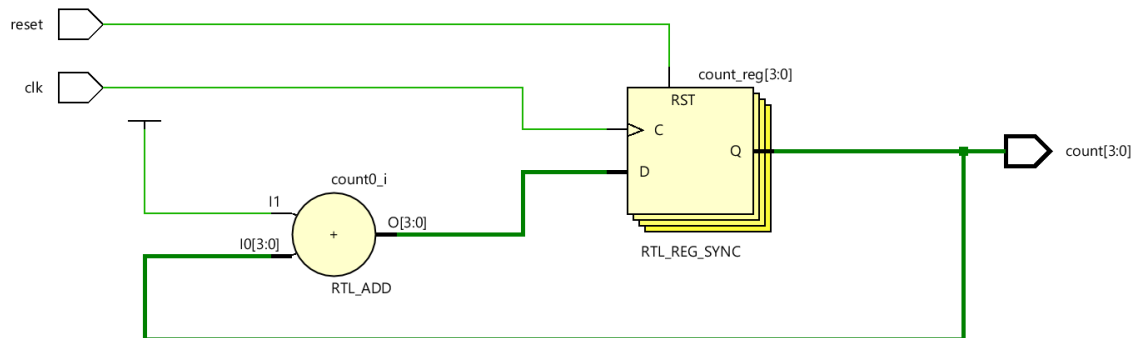
    fourbit_upcounter uut (.clk(clk),.reset(reset),.count(count));

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    initial begin
        reset = 1;
        #15 reset = 0;
        #250 reset = 1;
        #10 reset = 0;
        #30 $finish;
    end

    initial begin
        $monitor("time=%0t,reset=%b,clk=%b,count=%t", $time, reset, clk, count);
    end
endmodule

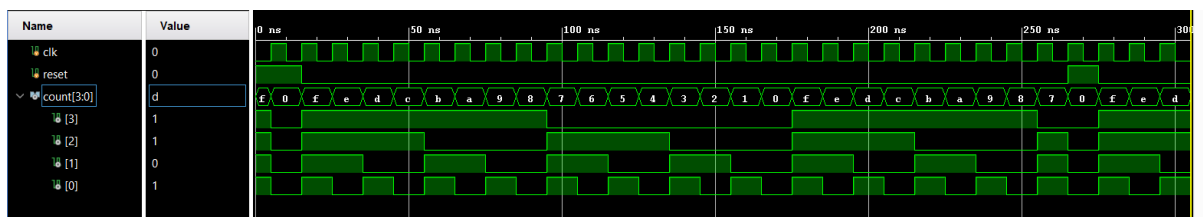
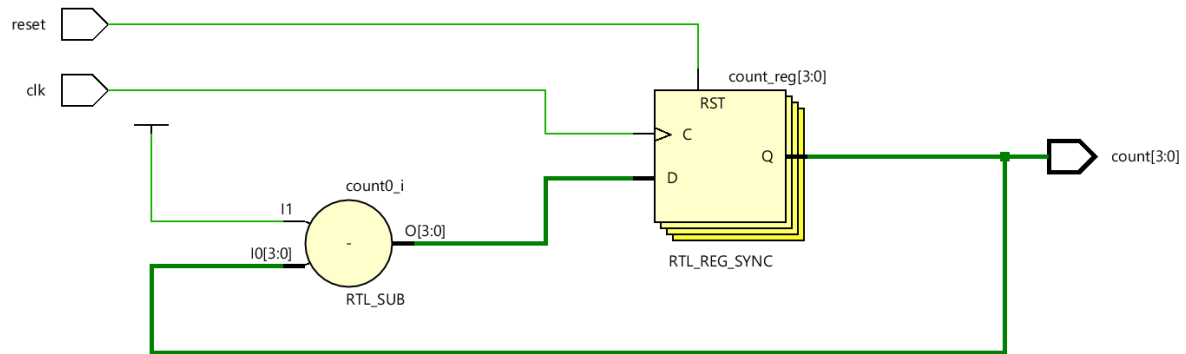
```



4-BIT DOWN COUNTER

```
module fourbit_downcounter(  
    input clk,  
    input reset,  
    output reg [3:0] count);  
  
    initial count = 4'b1111;  
  
    always @(posedge clk) begin  
        if (reset == 1'b1)  
            count <= 4'b0000;  
        else  
            count <= count -1;  
        end  
    endmodule
```

```
module fourbit_downcounter_TB;  
    reg clk, reset;  
    wire [3:0] count;  
  
    fourbit_downcounter uut (.clk(clk),.reset(reset),.count(count));  
  
    initial begin  
        clk = 0;  
        forever #5 clk = ~clk;  
    end  
  
    initial begin  
        reset = 1;  
        #15 reset = 0;  
        #250 reset = 1;  
        #10 reset = 0;  
        #30 $finish;  
    end  
  
    initial begin  
        $monitor("time=%0t,reset=%b,clk=%b,count=%t", $time, reset, clk, count);  
    end  
endmodule
```



4-BIT UP/DOWN COUNTER

```

module fourbit_upanddown_counter(
    input clk,
    input reset,
    input up_down,          // 1 for up, 0 for down
    output reg [3:0] count
);
    always @(posedge clk) begin
        if (reset)
            count <= 4'b0000;
        else if (up_down)
            count <= count + 1'b1; // Up count
        else
            count <= count - 1'b1; // Down count
    end
endmodule

```

```

module fourbit_upanddown_counter_TB;
    reg clk, reset, up_down;
    wire [3:0] count;

    fourbit_upanddown_counter uut
    (.clk(clk), .reset(reset), .count(count), .up_down(up_down));

```

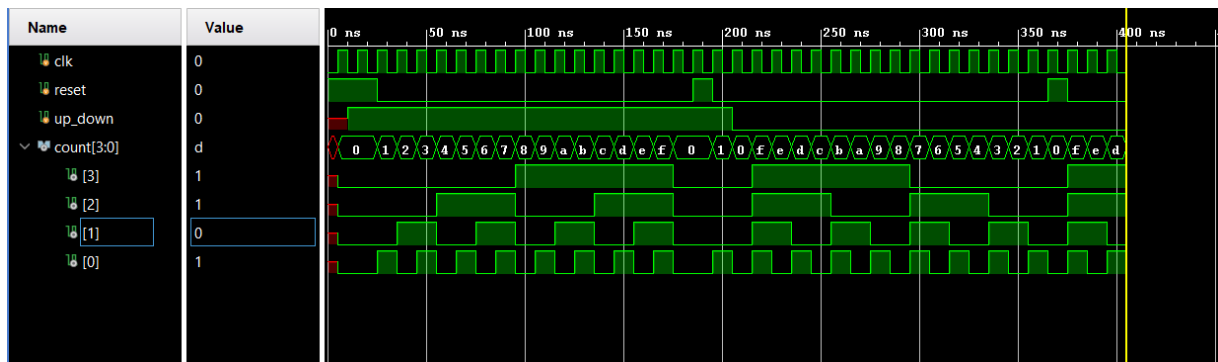
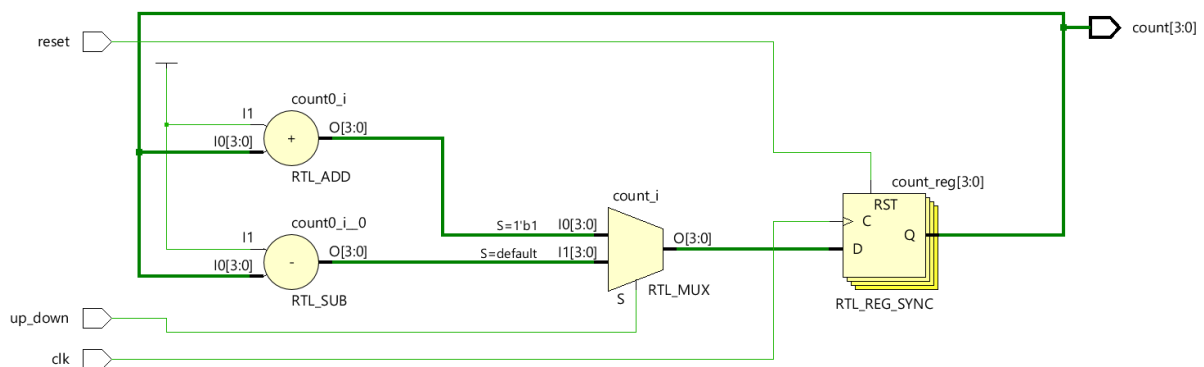
```

initial begin
    clk = 0;
    forever #5 clk = ~clk;
end

initial begin
    reset = 1;
    #10 up_down=1;
    #15 reset = 0;
    #160 reset = 1;
    #10 reset = 0;
    #10 up_down=0;
    #160 reset=1;
    #10 reset=0;
    #30 $finish;
end

initial begin
    $monitor("time=%0d | reset=%b | clk=%b | count=%d", $time, reset, clk,
count);
    end
endmodule

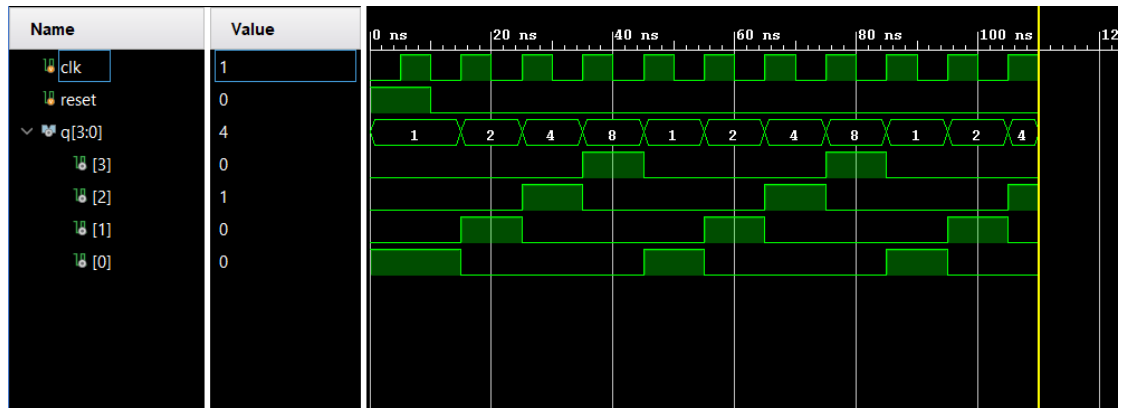
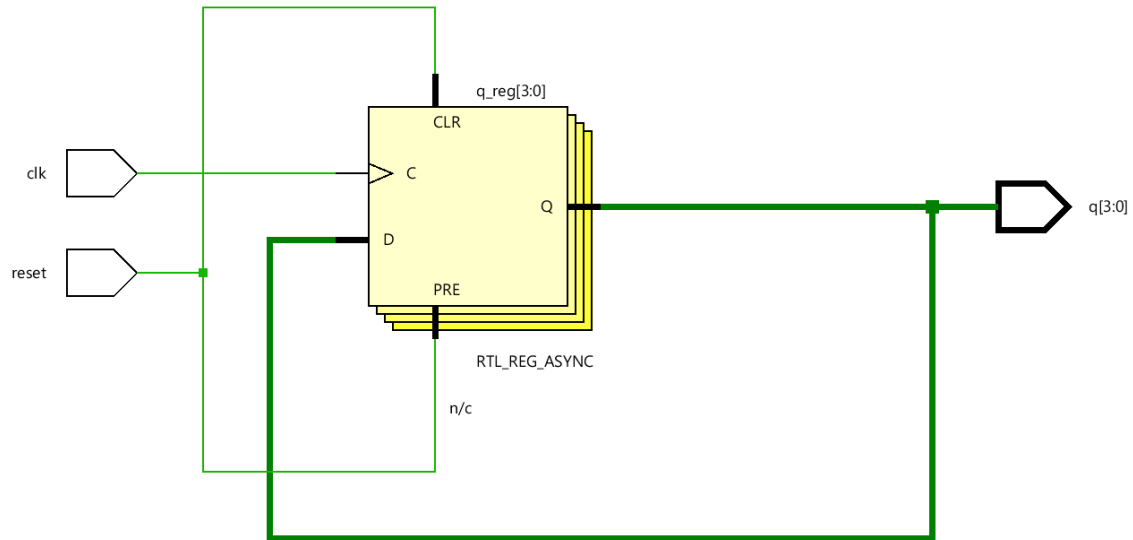
```



4-BIT RING COUNTER

```
module Ring_counter_4bit (input clk,reset,output reg [3:0] q);  
  
    always @(posedge clk or posedge reset) begin  
        if (reset)  
            q <= 4'b0001;  
        else  
            q <= {q[2:0], q[3]};  
        end  
    endmodule
```

```
module Ring_counter_4bit_TB;  
  
    reg clk;  
    reg reset;  
    wire [3:0] q;  
  
    Ring_counter_4bit uut (.clk(clk),.reset(reset),.q(q));  
  
    initial begin  
        clk = 0;  
        forever #5 clk = ~clk;  
    end  
  
    initial begin  
        $display("Time=%0t,reset=%b,q=%b",$time, reset, q);  
  
        reset = 1; #10;  
        reset = 0; #100;  
  
        $finish;  
    end  
endmodule
```



4-BIT JOHNSON COUNTER

```

module Johnson_counter_4bit (input clk,reset,output reg [3:0] q);

    always @(posedge clk or posedge reset) begin
        if (reset)
            q <= 4'b0000;
        else
            q <= {~q[0], q[3:1]};
        end
    endmodule

```



```

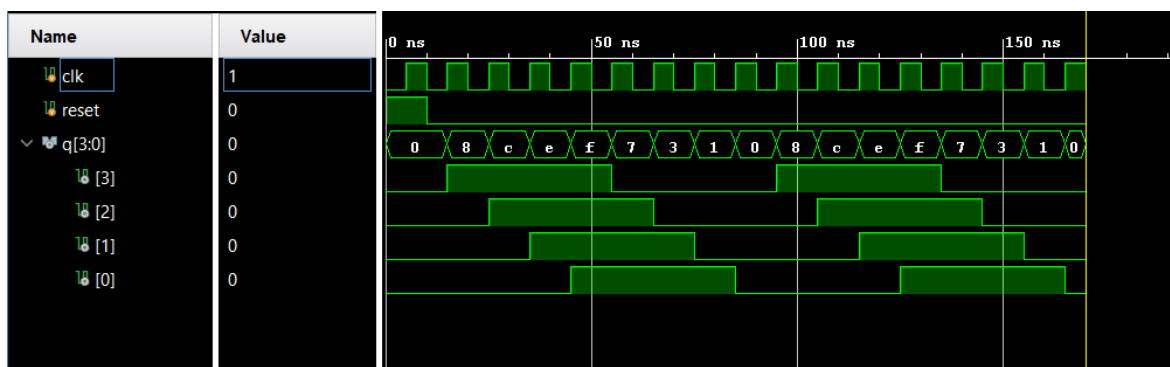
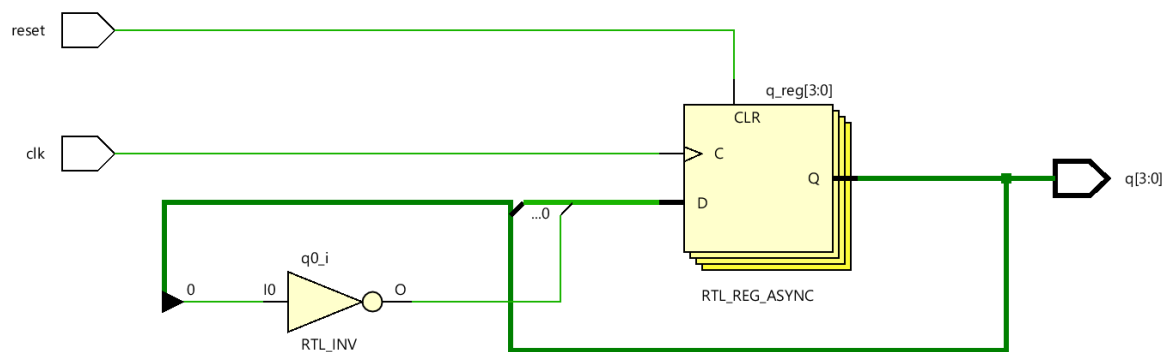
module Johnson_counter_4bit_TB;
    reg (clk,reset);
    wire [3:0] q;

    Johnson_counter_4bit uut (.clk(clk),.reset(reset),.q(q));

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end
    initial begin
        $display("Time=%0t,reset=%b,q=%b",$time,reset,q);
        reset = 1; #10;
        reset = 0; #160;
        $finish;
    end
end

endmodule

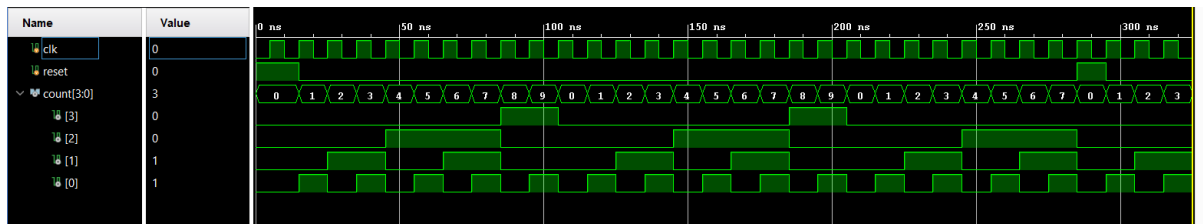
```



DECADE COUNTER

```
module Decade_counter(  
    input clk,  
    input reset,  
    output reg [3:0] count);  
  
    initial count = 4'b0000;  
  
    always @(posedge clk) begin  
        if (reset == 1'b1)  
            count <= 4'b0000;  
        else if (count == 4'b1001)  
            count <= 4'b0000;  
        else  
            count <= count + 1;  
        end  
  
    endmodule
```

```
module Decade_Counter_TB;  
    reg clk, reset;  
    wire [3:0] count;  
  
    Decade_counter uut (.clk(clk),.reset(reset),.count(count));  
  
    initial begin  
        clk = 0;  
        forever #5 clk = ~clk;  
    end  
  
    initial begin  
        reset = 1;  
        #15 reset = 0;  
        #270 reset = 1;  
        #10 reset = 0;  
        #30 $finish;  
    end  
  
    initial begin  
        $monitor("time=%0d | reset=%b | clk=%b | count=%d", $time, reset, clk,  
count);  
    end  
endmodule
```



4-BIT RIPPLE COUNTER

```
module four_bit_Ripple_counter_TB;
    reg clk, reset,t;
    wire [3:0] q;

    four_bit_ripple_counter uut(.clk(clk),.reset(reset),.t(t),.q(q));

    initial begin
```

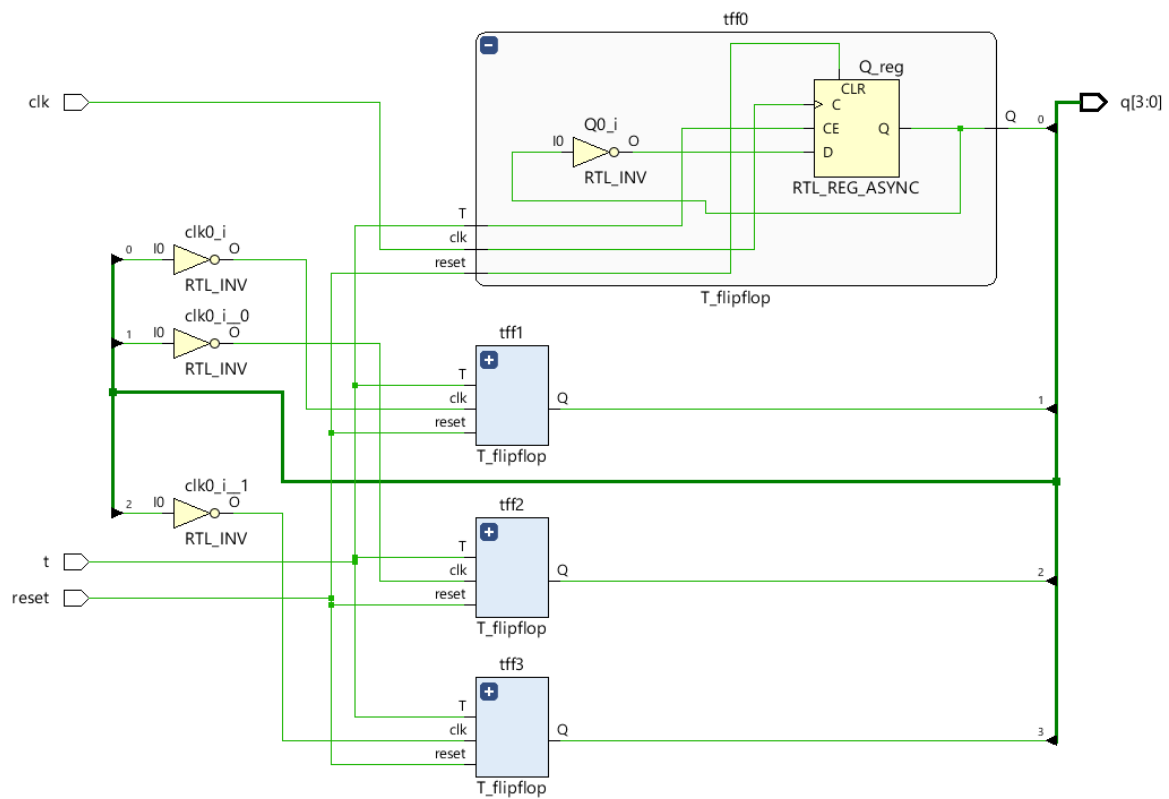
```

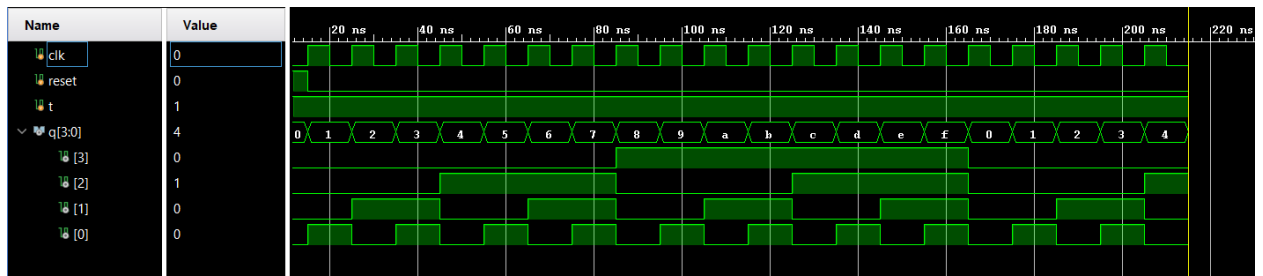
    clk = 0;
    forever #5 clk = ~clk;
end

initial begin
    t=1'b1;
end

initial begin
    reset = 1;
    #15 reset = 0;
    repeat(20) begin
        #10;
        $display("Time=%0t,reset=%b,q=%b,t=%b", $time,t,reset, q);
    end
    $finish;
end
endmodule

```





NUMERICALS

SUM OF NUMBERS

```

module sum_of_number(input[2:0]a,input [2:0]b,output reg[3:0] y);
function [3:0]sum_fun;
    input[2:0] a,b;
begin
    sum_fun = a+b;
end
endfunction
always @(*)begin
    y = sum_fun(a,b);
end
endmodule

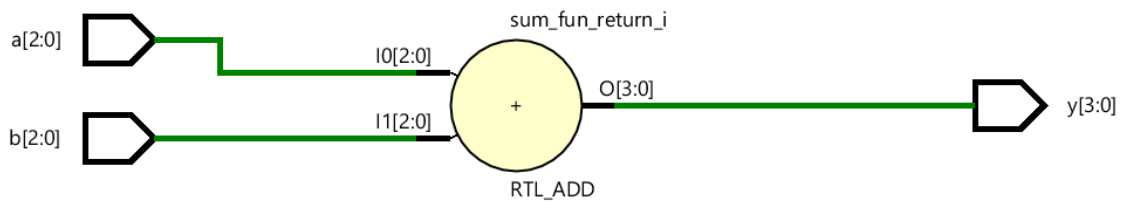
```

```

module sum_of_number_TB;
    reg [2:0]a,b;
    wire [3:0]y;

    sum_of_number uut (.a(a),.b(b),.y(y));
    initial
    begin
        $monitor ("Time =%0t, a=%d(%b),b=%d(%b),y=%d(%b)", $time ,a,a,b,b,y,y);
        a = 3'd0; b = 3'd0; #10;
        a = 3'd1; b = 3'd2; #10;
        a = 3'd3; b = 3'd4; #10;
        a = 3'd5; b = 3'd2; #10;
        a = 3'd7; b = 3'd7; #10;
        $finish;
    end
endmodule

```



Name	Value	0 ns	5 ns	10 ns	15 ns	20 ns	25 ns	30 ns	35 ns	40 ns	45 ns	50 ns
> a[2:0]	7	0		1		3		5		7		
> b[2:0]	7	0		2		4		2		7		
> y[3:0]	e	0		3				7			e	

SQUARE OF NUMBER

```

module square_number(input[2:0] A,output reg [15:0] Y);
function [15:0]squ_number (input [2:0]b);
begin
squ_number=b*b;
end
endfunction

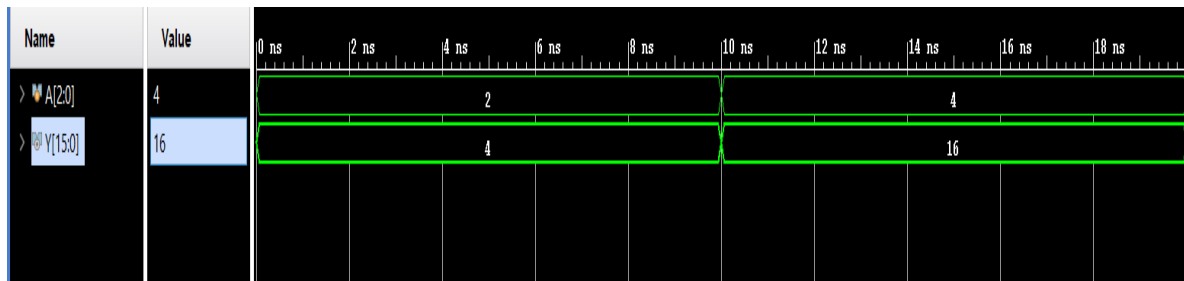
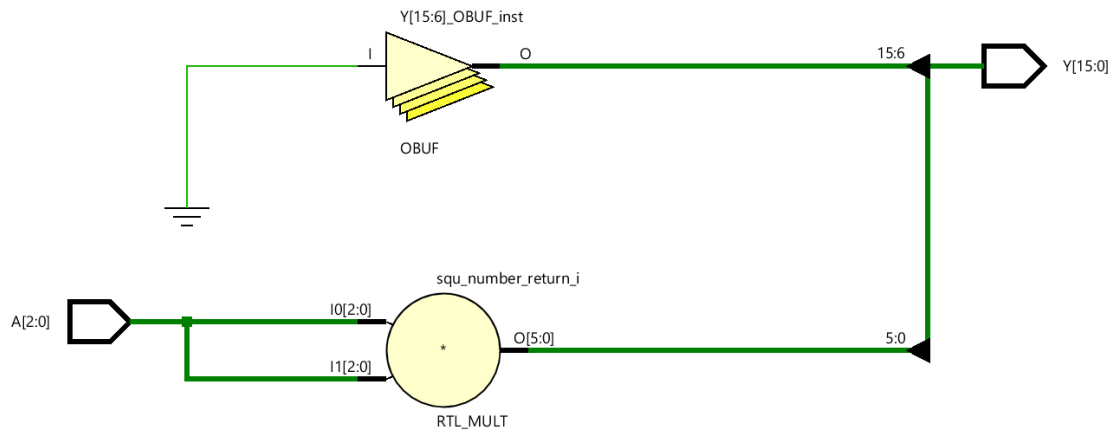
always @(*)
Y=squ_number(A);
endmodule

```

```

module square_of_number_TB;
reg [2:0]A;
wire [15:0]Y;
square_number dut (.A(A),.Y(Y));
initial
begin
A=2;
#10 A=4;
#10 $finish;
end
endmodule

```



FACTORIAL OF A NUMBER

```

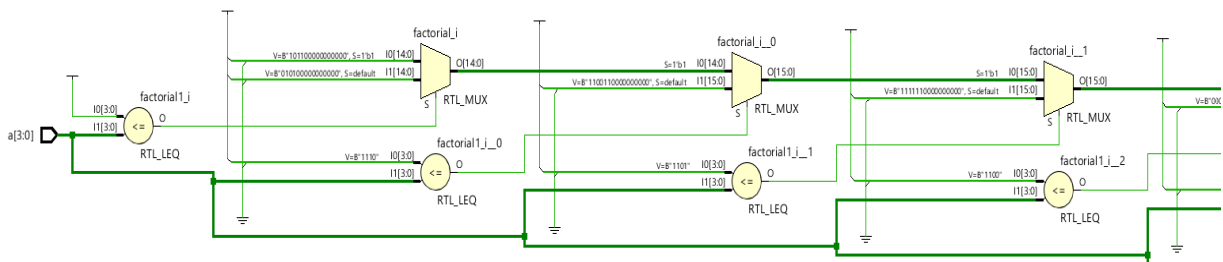
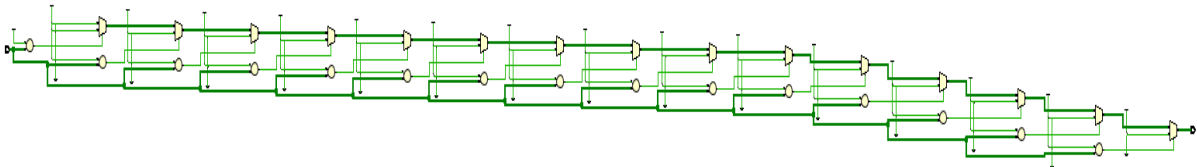
module factorial_of_num_fun(a,y);
input [3:0]a;
output reg [15:0]y;
function [15:0] factorial;
input [3:0]n;
integer i ;
begin
factorial=1;
for (i=1;i<=n;i=i+1)
    factorial = factorial*i;
end
endfunction
always @(*)
begin
y=factorial(a);
end
endmodule

```

```

module factorial_of_fun_TB;
reg [3:0] a;
wire [15:0]y;
factorial_of_num_fun dut (.a(a),.y(y));
initial
begin
a=4'b0101; #20
a=4'b0100; #20
$finish;
end
endmodule

```



Name	Value	0 ns	5 ns	10 ns	15 ns	20 ns	25 ns	30 ns	35 ns	40 ns
> a[3:0]	4	5				4				
> y[15:0]	24	120				24				

FIBANOCCHI SERIES

```
module Fibonacci_series(input clk,reset,output reg [31:0] fib);

    reg [31:0] prev;
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            fib <= 32'd0;
            prev <= 32'd1;
        end else begin
            fib <= fib + prev;
            prev <= fib;
        end
    end
endmodule
```

```
module fibonacci_tb;
    reg clk, reset;
    wire [31:0] fib;

    Fibonacci_series uut (.clk(clk),.reset(reset),.fib(fib));

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end
    initial begin
        reset = 1;
        #15 reset = 0;
        repeat (20) begin
            #10;
            $display("Time=%0t,fib= %d", $time, fib);
        end
        $finish;
    end
endmodule
```

